

C# fundamentals

Introduction to .NET Full stack development

ASP Core

Prerequisites:

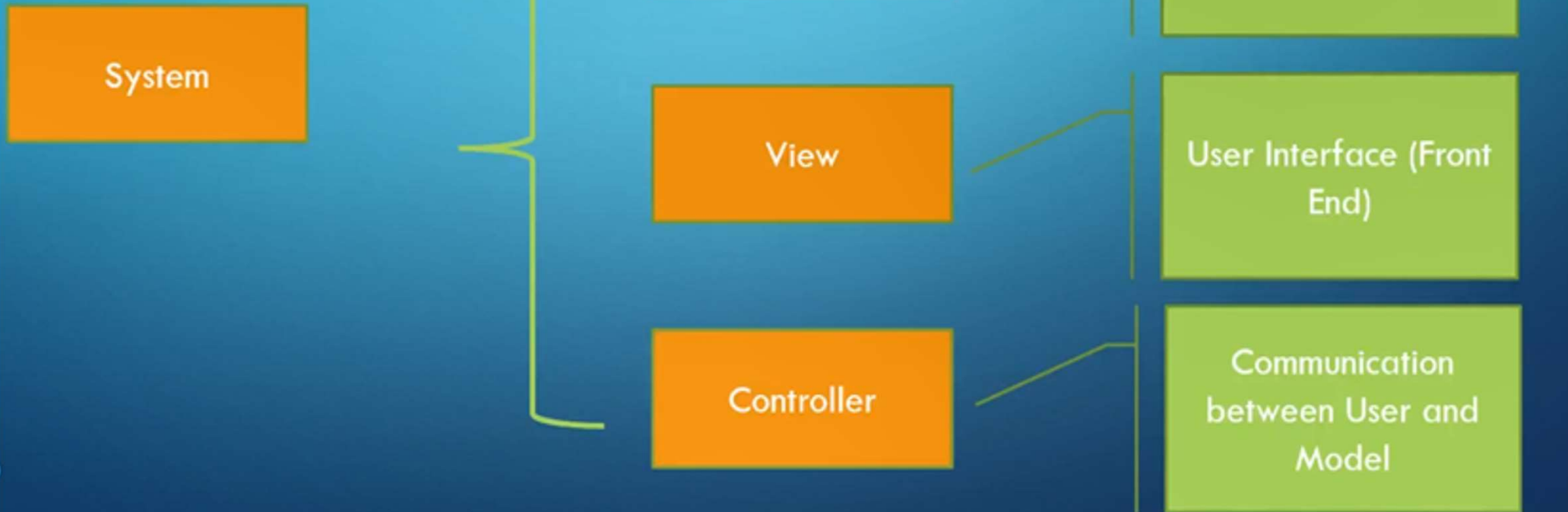
- **C#**
- **OOP**
- **Principles**
- **SQL Server**
- **LINQ**
- **EF**

ASP Core

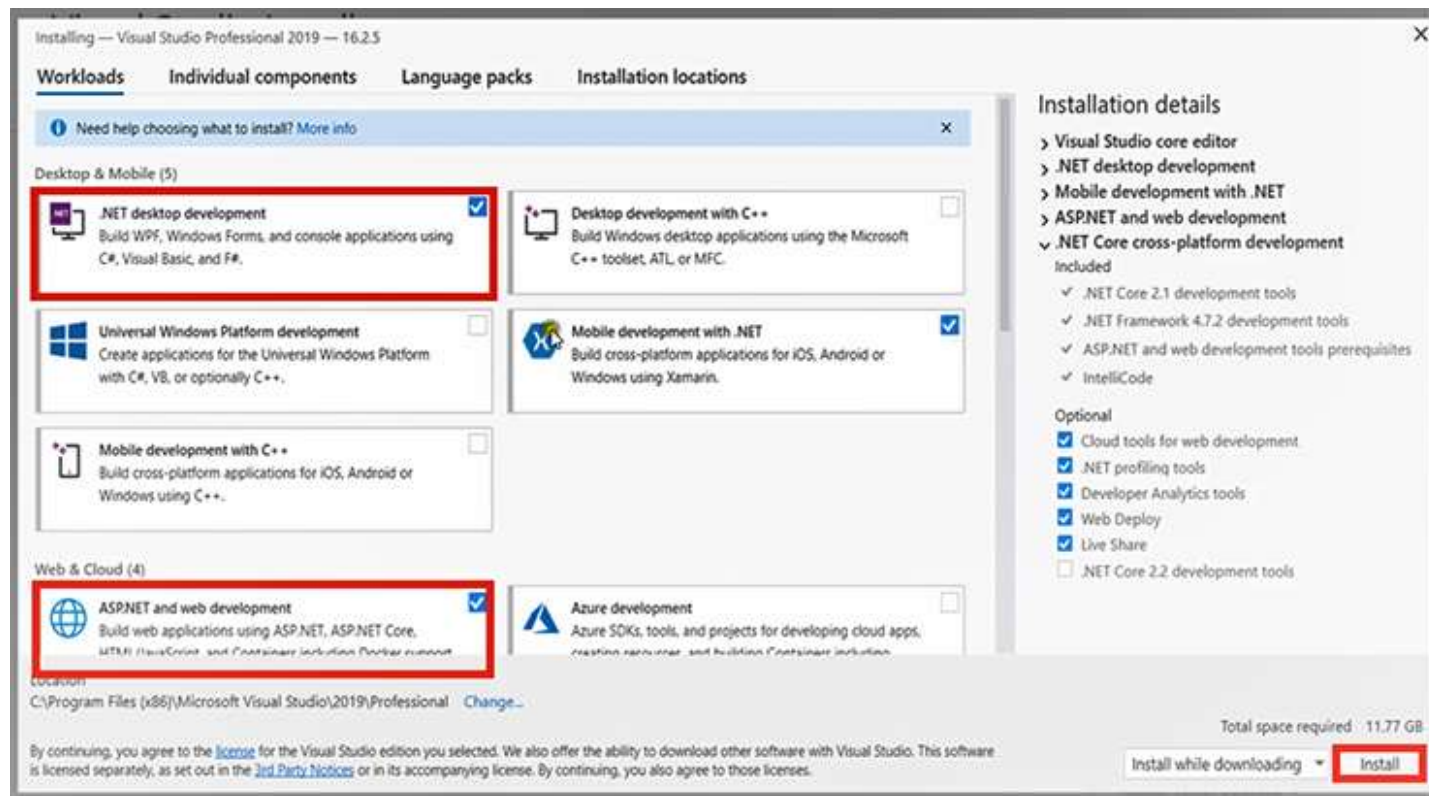
What's ASP. Net Core?

- Asp.net is a popular web- development framework for building we apps on the .Net platform.
- Asp.net Core is the open source version of asp.net, that runs on macOS, Linux, and Windows.
- Asp.Net Core was first released in 2016.

- ASP.NET MVC is an **excellent framework** which provides ultimate support for the pattern of **MVC design** to



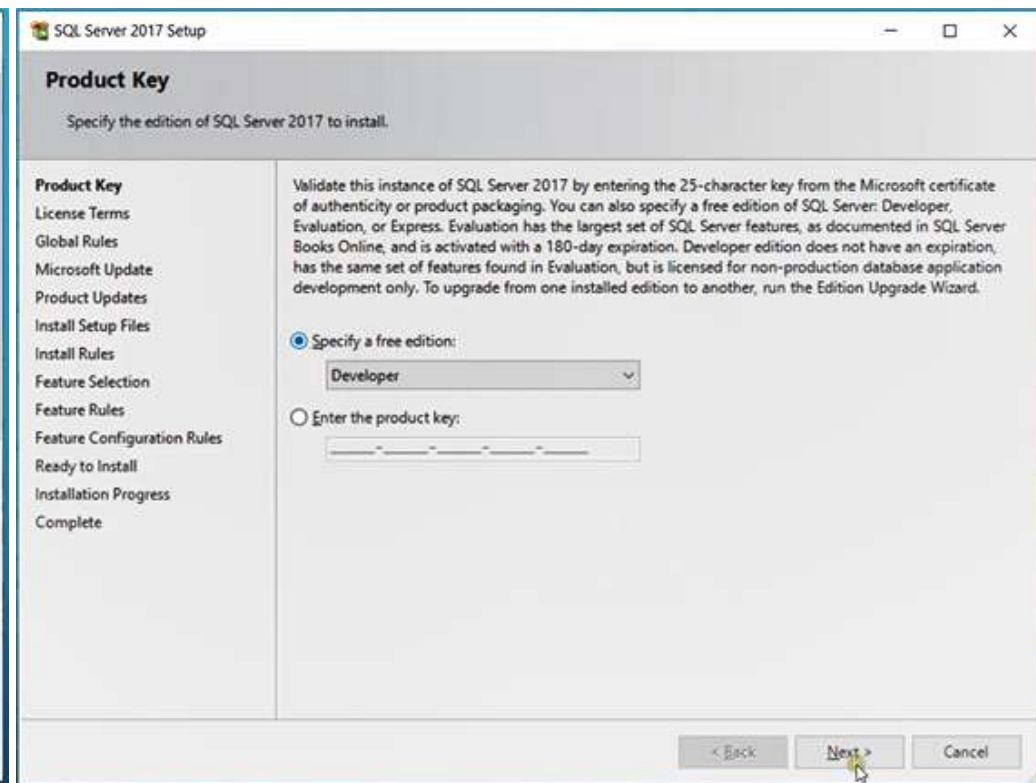
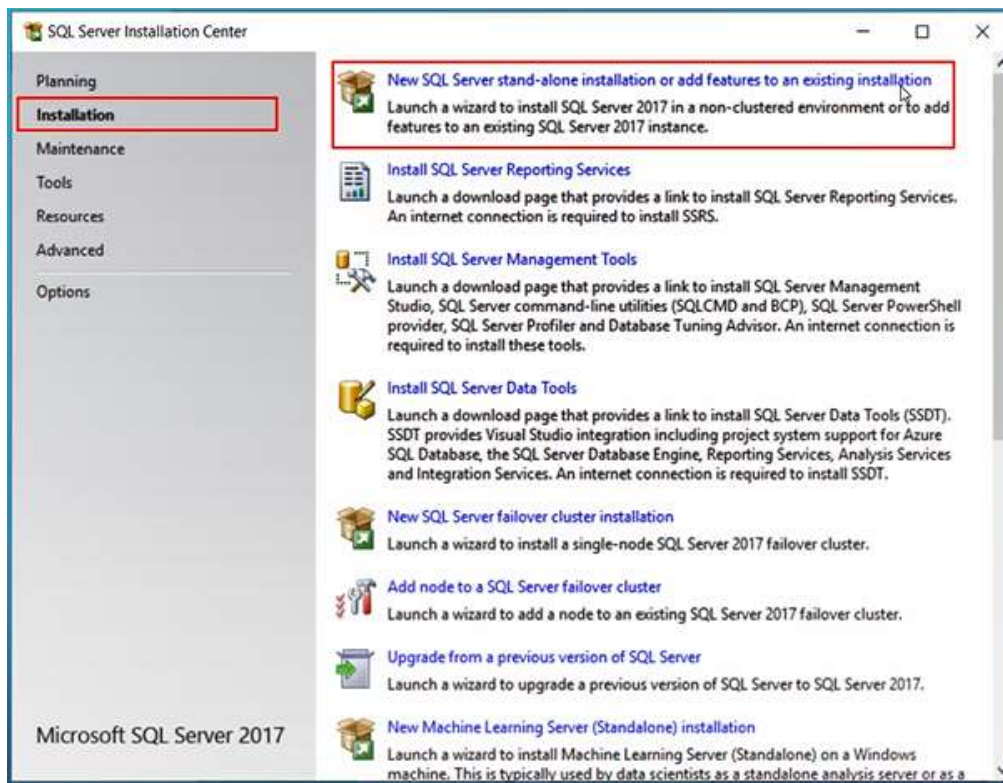
Installation



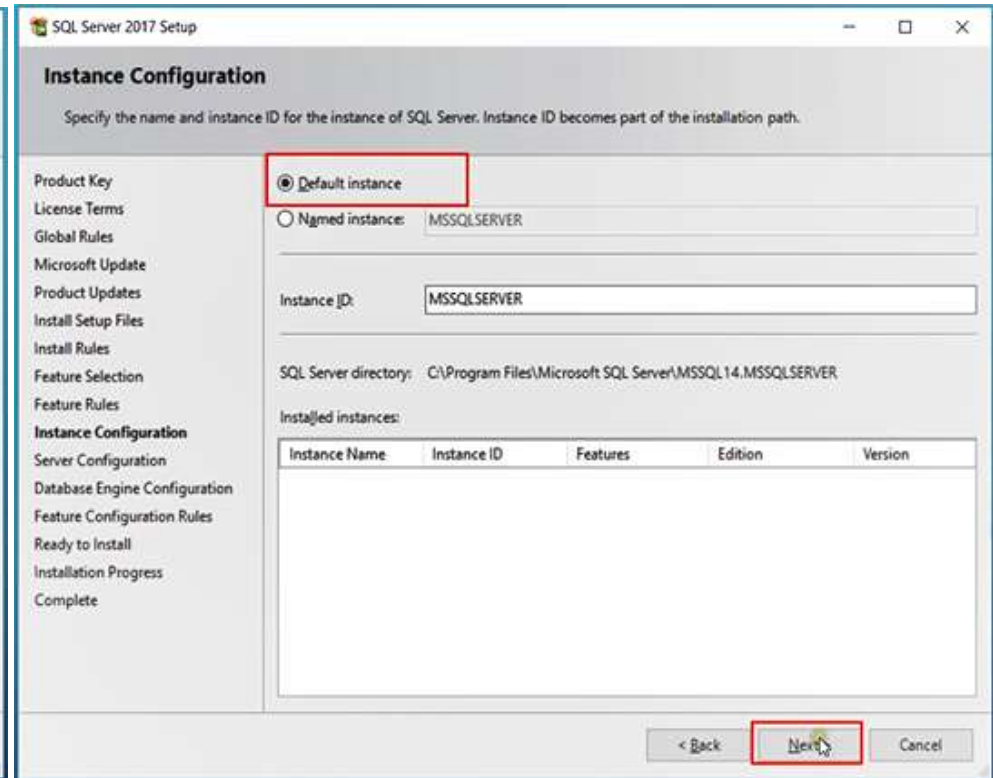
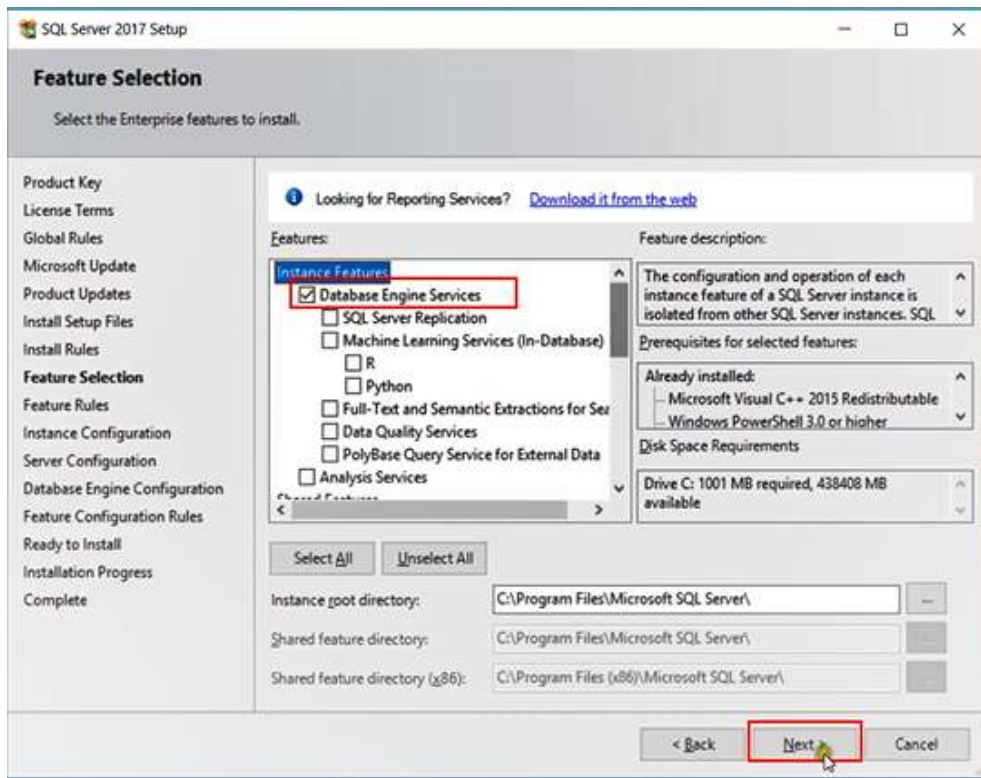
Installation



Installation



Installation



Installation

SQL Server 2017 Setup

Database Engine Configuration

Specify Database Engine authentication security mode, administrators, data directories and TempDB settings.

Product Key
License Terms
Global Rules
Microsoft Update
Product Updates
Install Setup Files
Install Rules
Feature Selection
Feature Rules
Instance Configuration
Server Configuration
Database Engine Configuration
Feature Configuration Rules
Ready to Install
Installation Progress
Complete

Server Configuration | Data Directories | TempDB | FILESTREAM

Specify the authentication mode and administrators for the Database Engine.

Authentication Mode

☒ Windows authentication mode

☐ Mixed Mode (SQL Server authentication and Windows authentication)

Specify the password for the SQL Server system administrator (sa) account.

Enter password:

Confirm password:

Specify SQL Server administrators

SCI-000969-RESE\pc (pc)

SQL Server administrators have unrestricted access to the Database Engine.

Add Current User Add... Remove

< Back Next > Cancel

SQL Server Installation Center

Planning
Installation
Maintenance
Tools
Resources
Advanced
Options

New SQL Server stand-alone installation or add features to an existing installation
Launch a wizard to install SQL Server 2017 in a non-clustered environment or to add features to an existing SQL Server 2017 instance.

Install SQL Server Reporting Services
Launch a download page that provides a link to install SQL Server Reporting Services. An internet connection is required to install SSRS.

Install SQL Server Management Tools
Launch a download page that provides a link to install SQL Server Management Studio, SQL Server command-line utilities (SQLCMD and BCP), SQL Server PowerShell provider, SQL Server Profiler and Database Tuning Advisor. An internet connection is required to install these tools.

Install SQL Server Data Tools
Launch a download page that provides a link to install SQL Server Data Tools (SSDT). SSDT provides Visual Studio integration including project system support for Azure SQL Database, the SQL Server Database Engine, Reporting Services, Analysis Services and Integration Services. An internet connection is required to install SSDT.

New SQL Server failover cluster installation
Launch a wizard to install a single-node SQL Server 2017 failover cluster.

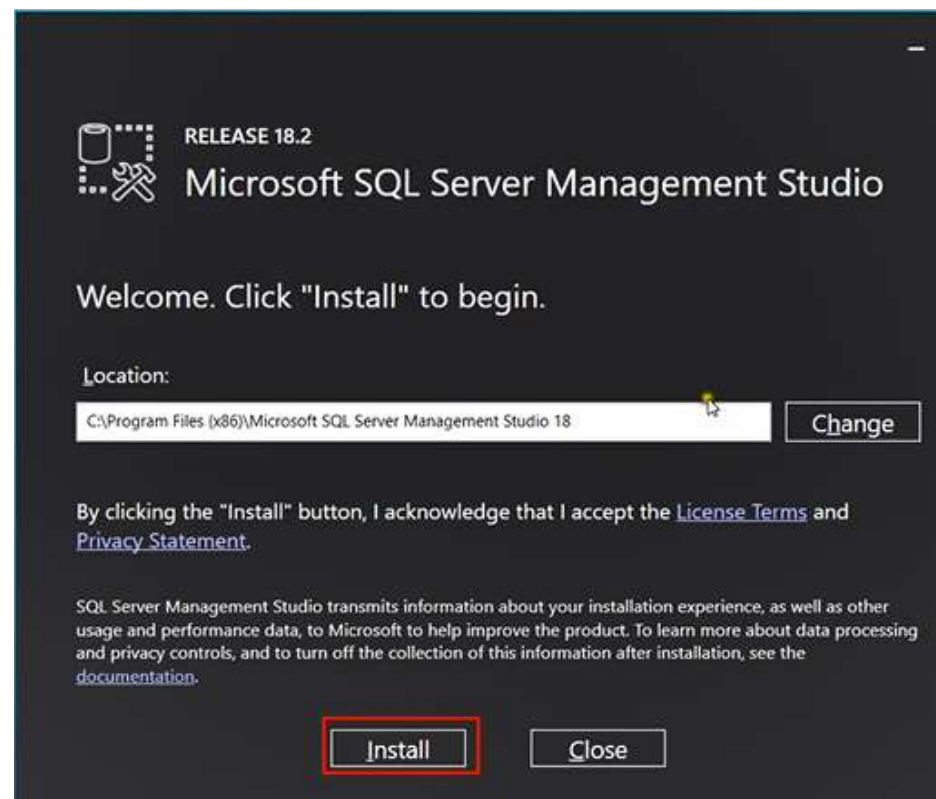
Add node to a SQL Server failover cluster
Launch a wizard to add a node to an existing SQL Server 2017 failover cluster.

Upgrade from a previous version of SQL Server
Launch a wizard to upgrade a previous version of SQL Server to SQL Server 2017.

New Machine Learning Server (Standalone) installation
Launch a wizard to install Machine Learning Server (Standalone) on a Windows machine. This is typically used by data scientists as a standalone analysis server or as a

Microsoft SQL Server 2017

Installation



Why .NET Core?

Cross Platform:

Easier for developers to create applications that can be used on multiple platforms.

Performance:

.NET Core is built with performance in mind.

Cloud-ready:

.NET Core is designed to work well in cloud environments.

Modern development:

.NET Core provides developers with modern development tools and techniques.

ASP Core

What's API?

API is acronym for Application Programming Interface which is a mechanisms that enable two software components to communicate with each other using a set of definitions and protocols.

APIs Should:

Be stateless

Respect HTTP methods.

Respect HTTP standard status codes.

- ليس مقتصرة على الويب
- في سياق الويب تمثل ويب سيرفيس

ASP Core

What's HTTP?

- HTTP is acronym for Hypertext Transfer Protocol which is the protocol that enables the transfer for data over internet, allowing users to access websites and other online resources.



HTTP Methods/Verbs

Method	Usage
GET	Used to get data from API
POST	Used to create new records
PUT	Used to update existing records
DELETE	Used to delete records

HTTP Status Codes

Method	Usage
200-299	Successful response
300-399	Redirection
400-499	Client errors
500-599	Server errors

Most Common HTTP Status Codes

Code	Name	Usage
200	OK	Successful response
201	Created	Resource has been created
204	No Content	Request succeeded but no content returned from the server
400	Bad Request	Request is not processed because it's not valid
401	Unauthorized	The client is not <u>authenticated</u>
403	Forbidden	The client is authenticated but doesn't have permission to perform the request
404	Not Found	The requested resource was not found
405	Method Not Allowed	The request method is not supported by the target resource
409	Conflict	The request conflicts with the current state

ASP Core

What's JSON?

JSON is acronym for JavaScript Object Notation, is open standard format, which is lightweight and text-based, designed explicitly for human-readable data interchange.

```
public class Person
{
    public int Id { get; set; }
    public string Name { get; set; }
    public DateTime BirthDate { get; set; }
    public Address Address { get; set; }
}

public class Address
{
    public string City { get; set; }
    public string Area { get; set; }
    public string BuildingNumber { get; set; }
    public string ApartmentNumber { get; set; }
}
```

```
{
  "Person": {
    "Id": 0,
    "Name": "string",
    "BirthDate": "1/1/0001 12:00:00 AM",
    "Address": {
      "City": "string",
      "Area": "string",
      "BuildingNumber": "string",
      "ApartmentNumber": "string"
    }
  },
  "Address": {
    "City": "string",
    "Area": "string",
    "BuildingNumber": "string",
    "ApartmentNumber": "string"
  }
}
```

Installing visual studio

- In order to code in C#, we need to install the latest stable release of Microsoft Visual Studio.
- Which is an integrated development environment (IDE) from Microsoft.
- First, visit the following Visual Studio free download link.
 - <https://visualstudio.microsoft.com/downloads/>
- It used to code in C# and develop computer programs, websites, web apps, mobile apps, etc.

Asp.net MVC Routing – URL Pattern

URL	Controller	Action	Id
http://localhost/home	HomeController	Index	null
http://localhost/home/index/123	HomeController	Index	123
http://localhost/home/about	HomeController	About	null
http://localhost/home/contact	HomeController	Contact	null

Controller Action method
Id parameter value

http://localhost:1234/home/index/100

Controller Action method

http://localhost:1234/home/index

Ex:

```
public string str()  
{  
    return "fun to learning MVC.Net";  
}
```

```
public int Add()  
{  
    int a = 8, b = 10;  
    return a + b;  
}
```

Ex:

```
public RedirectResult ShowInternal()  
{  
    return Redirect("/Home/Index");  
}  
public RedirectResult ShowExternal()  
{  
    return Redirect("https://www.google.com/");  
}
```



```
Index.cshtml* - HomeController.cs -  
4      }  
5  
6      <!DOCTYPE html>  
7  
8      <html>  
9      <head>  
10         <meta name="viewport" content="width=device-width" />  
11         <title>Index</title>  
12     </head>  
13     <body>  
14         <div>  
15             <h1>Wellcome to series MVC5</h1>  
16         </div>  
17     </body>  
18 </html>
```

Example

```
public class HomeController : Controller
```

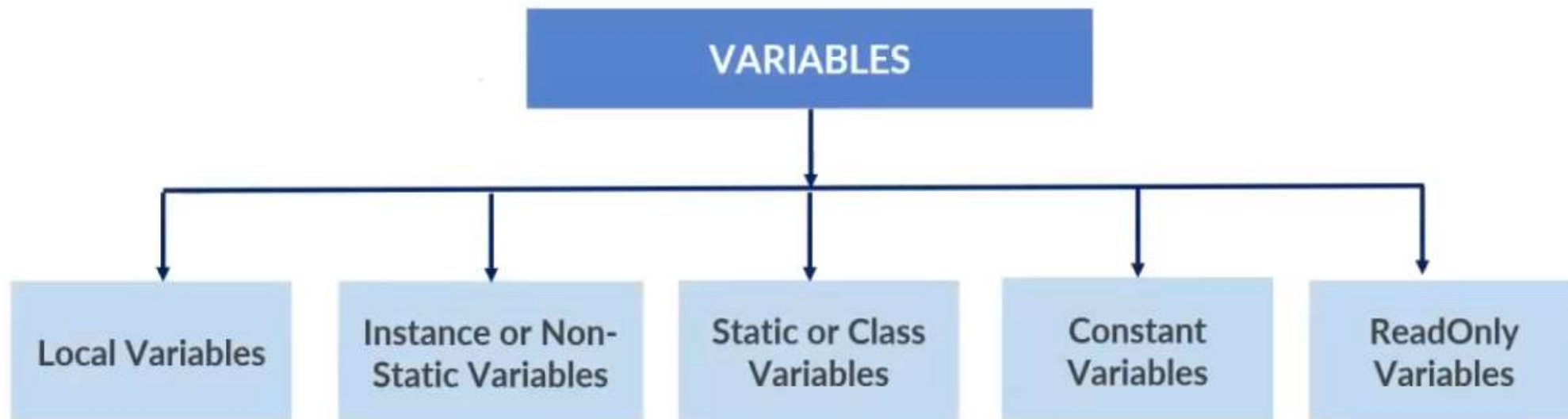
```
public ActionResult Details()
{
    int x = Convert.ToInt32(HttpContext.Request.Query["id"]);
    string country = "";
    switch (x)
    {
        case 1: country = "Germany"; break;
        case 2: country = "Italy"; break;
        case 3: country = "France"; break;
        case 4: country = "Syria"; break;
        default: country = "Sorry, you do not belong to any country."; break;
    }
    return Content(country);
}
```

```
public class TestController : Controller
```

```
{  
    public ActionResult List()  
    {  
        return View();  
    }  
}
```

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Details</th>
  </tr>
  <tr>
    <td>Osama</td>
    <td><a href="/Home/Details?id=1">Details</a></td>
  </tr>
  <tr>
    <td>Sami</td>
    <td><a href="/Home/Details?id=2">Details</a></td>
  </tr>
  <tr>
    <td>Samer</td>
    <td><a href="/Home/Details?id=3">Details</a></td>
  </tr>
  <tr>
    <td>Abeer</td>
    <td><a href="/Home/Details?id=4">Details</a></td>
  </tr>
</table>
```

TYPES OF VARIABLES



// Expression Lambda:

```
var numbers = new int[] { 2, 5, 6, 5, 1, 3, 5, 7 };  
var count = numbers.Count(x => x == 5);  
Console.WriteLine(count);
```

// Statement Lambda

```
List<int> numbers2 = new List<int> { 2, 5, 6, 5, 1, 3, 5, 7 };  
count = numbers.Count(x => { return x == 5; });  
Console.WriteLine(count);
```

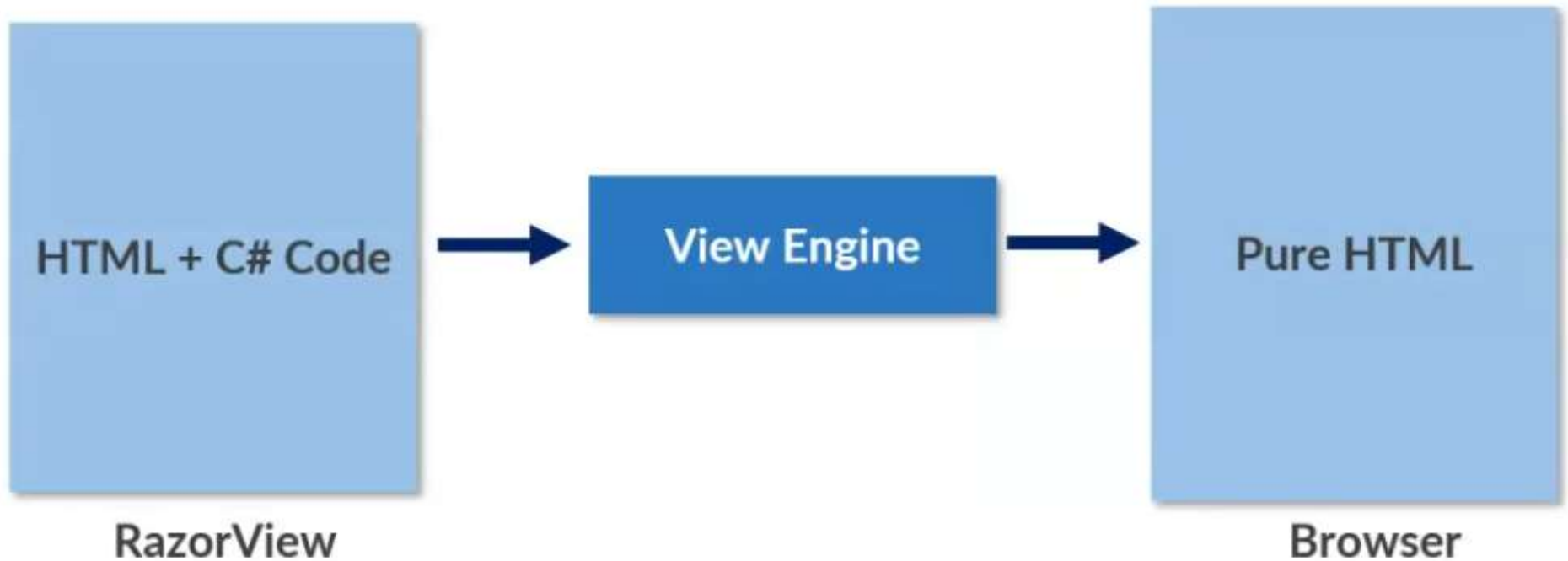

Razor View Engine

- It is responsible for producing HTML response when invoked by controller Action method.
- Controller action methods return some types of responses, which are called as Action Results.
- The viewResults are produced by the View Engine.
- When the controller Action method invokes the view() or PartialView(),
- It invokes the view engine, which produces the HTML Response.

Razor View Engine

- The razor View engine is the default view engine for the ASP.NET Core apps.
- It looks for Razor markup in the View File and Parses it and produces the HTML response.
- It is used to generate views that are returned to the client's browser in response to a request.
- It is a powerful and flexible templating engine.
- With this. It is easier to create dynamic web pages in asp.net Core.
- It enables developers to write clean and maintainable code that can be easily understood.

Razor View Engine



Razor Syntax

- Razor is markup syntax for embedding .NET based code into webpages.
- The Razor syntax consists of Razor markup, C#, and HTML.
- Files containing Razor generally have a .cshtml file extension
- Razor syntax is similar to the templating engines of various JavaScript single-page application (SPA) frameworks, such as Angular, React etc.
- Razor supports C# and uses the @ symbol to transition from HTML to C#.
- When an @ symbol is following by a razor reserved keyword.

Razor Syntax

- To escape an @ symbol in Razor markup, use a second @ symbol:

```
<p>@@Username</p>
```

- The code is rendered in HTML with a single @ symbol:

```
<p>@Username</p>
```

```
public class HomeController : Controller
```

```
public IActionResult Index()
```

```
{
```

```
    ViewBag.message = "Omar Suleiman";
```

```
}
```

```
<div>
```

```
    <p>@ViewBag.message</p>
```

```
</div>
```

```
public class Customer
{
    public int Id { get; set; }
    public string ?Name { get; set; }
    public double Amount { get; set; }
}
```

```
public static List<Customer> customers = new List<Customer>()
{
    new Customer(){Id=101, Name="Smith", Amount=12000},
    new Customer(){Id=102, Name="Jone", Amount=15000}
};

public IActionResult Index()
{
    ViewBag.CustomerCount = customers.Count();
    ViewBag.CustomerList = customers;
    return View();
}
```



```
<ul>  
    @foreach (var item in ViewBag.CustomerList)  
    {  
        <li>@item.Id - @item.Name - @item.Amount</li>  
    }  
</ul>
```

Layout View

- the layouts are like the **master** pages in webforms applications.
- A layout view defines the common layout or structure of multiple views in an application.
- It allows you to define a consistent layout or structure for your application.
- Such as a header, footer, and navigation menu, which can be reused across multiple views.
- It is used to define the structure of the HTML page and may contain placeholders for the content that will be displayed by the views.

Use Of Layout View

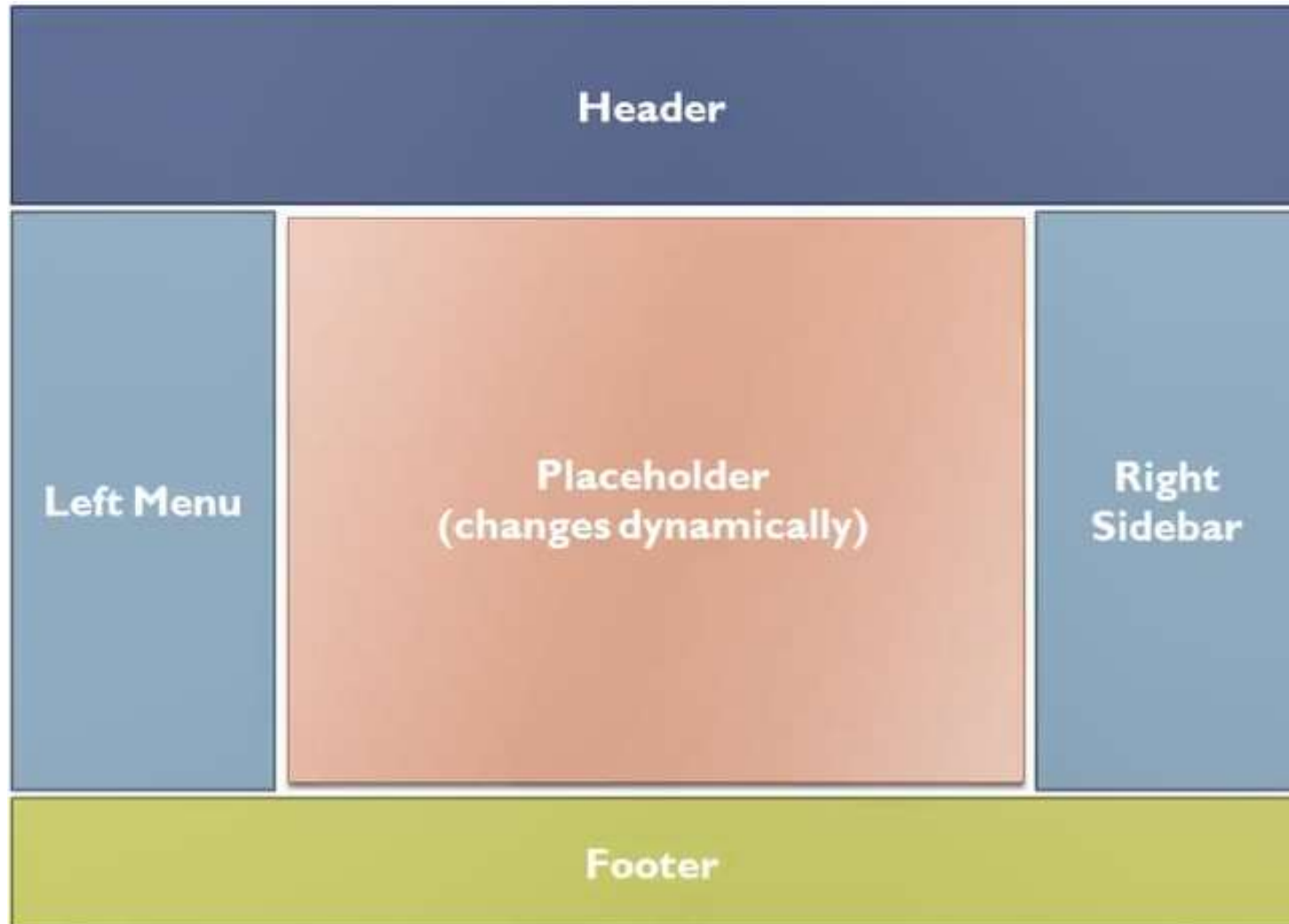
Nowadays, almost all web applications have the following sections:

- Website Header
 - Website Footer
 - Navigation Menus
 - Main Content Area
- Instead of putting all the sections (i.e. the HTML) in each and every view pages.
 - It is always better and advisable to put them in a layout view.
 - And them inherit that layout view in each and every view where you want that look and feel.
 - With layout views, now it is easier to maintain the consistent look and feel of your application.

Use Of Layout View



Layout View



HTML Helper

- The HTMLHelper class renders HTML controls in the razor view.
- It binds model object to HTML controls to display value of model properties into those controls.
- and also assigns the value of the controls to the model properties while submitting a web form.
- So always use the HTMLHelper class in razor view instead of writing HTML tags manually.
- The difference between calling the HtmlHelper methods and using as HTML tags is that
 - the HtmlHelper method is designed to make it easy to bind to view data or model data.

HTML HELPER

- The following table lists the HtmlHelper methods and HTML control each method renders.

Extension Method	Strongly typed Method	HTML Control
Html.ActionLink()	NA	<a>
Html.TextBox()	Html.TextBoxFor()	<input type="textbox">
Html.TextArea()	Html.TextAreaFor()	<input type="textarea">
Html.CheckBox()	Html.CheckBoxFor()	<input type="checkbox">
Html.RadioButton()	Html.RadioButtonFor()	<input type="radio">
Html.DropDownList()	Html.DropDownListFor()	<select> <option> </select>
Html.ListBox()	Html.ListBoxFor()	multi-select list box: <select>
Html.Hidden()	Html.HiddenFor()	<input type="hidden">

Index in HomeController.cs

```
<hr />  
@Html.ActionLink("DetailsPage", "Add", "Test");
```

```
@* Comment *@  
@* @Html.ActionLink("linkText","actionName","routeValues"); *@
```

In TestController.cs

```
public int Add()  
{  
    int a = 8, b = 10;  
    return a + b;  
}
```


Tag Helper

- Tag Helper is a new feature in ASP.NET MVC
- It enables server-side code to create and render HTML elements in Razor View.
- It is a feature of Razor View Engine.
- They are the C# classes which participate in view generation by creating the HTML elements.
- We can change the content of HTML elements and additional attributes to HTML element.
- It is very similar to HTML helper in ASP.NET MVC.

Tag Helper

- HTML HELPER:

```
@Html.TextBoxFor(model => model.Name, new { @class =  
    "form-control", placeholder = "Enter Your Name" })
```

- EQUIVALENT WITH TAG HELPER:

```
<input asp-for="Name" placeholder="Enter Your Name"  
class="form-control" />
```

- EQUIVALENT HTML:

```
<input placeholder="Enter Your Name" class="form-  
control" id="Name" name="Name" value="" type="text">
```

```
<hr />
```

```
@Html.ActionLink("Message", "Add", "Test");
```

```
<br />
```

```
<a asp-controller="Test" asp-action="Add">Message</a>
```

Convention Routing

- It is used for defining routes based on conventions and defaults,
- Rather than explicitly defining each route.
- Developers define a set of conventions that are used to generate routes for different types of controllers and actions,
- It provides a simple and flexible way to define routes in an ASP.NET Core application.
- The URL pattern is generated based on the names of the controller and action methods.

CONVENTION ROUTING

- For example, consider the following code:

```
app.UseMvc(routes =>
{
    routes.MapRoute(
        name: "default",
        template: "{controller}/{action}/{id?}",
        defaults: new { controller = "Home", action = "Index" });
});
```

- This code sets up a default route that maps URLs in the format of `/[controller]/[action]/[id]`.

```
[Route("admin/signin")]
public ActionResult Login()
{
    return View();
}

public ActionResult Validate(IFormCollection values)
{
    string un = Convert.ToString(values["uname"]);
    string pw = Convert.ToString(values["pass"]);
    if (un == "scott" && pw == "tiger")
        return Content("welcome");
    else
        return Content("sorry!");
}
```

```
<h2>Login</h2>
```

```
<hr />
```

```
<form method="post" action="/Home/Validate">
```

```
    <label>Enter Username</label><br />
```

```
    <input type="text" name="uname" /><br />
```

```
    <label>Enter Password</label><br />
```

```
    <input type="password" name="pass" /><br />
```

```
    <input type="submit" value="Login" /><br />
```

```
</form>
```

< > ↻ 🔒 localhost:7244/admin/signin

 Inbox (1) - affinity.s...  NewHorizon

WebApplication1 [Home](#) [Privacy](#)

Login

Enter Username

Enter Password

Login

Attribute Routing

- you can define routes at controller level which apply all actions
- within the controller unless a specific route is add to an action.
- it avoids repeating Route attribute value for each action
- method that maps to a URL under the same controller.
- `RoutePrefix` is the prefix for any route that we to apply



```
RouteConfig.cs | Register.cshtml | AccountController.cs* | Login.cshtml
Ex4.Controllers.AccountController

using System.Web.Mvc;

namespace Ex4.Controllers

[RoutePrefix("admin")]
0 references
public class AccountController : Controller
{
    // GET: Account
    0 references
    public ActionResult Index()...
    [Route("signup")]
    0 references
    public ViewResult Register()...
    [Route("reset-password")]
    0 references
    public ViewResult ChangePassWord()...
```

Action Level Attribute Routing

- Route values or Route parameter can be added to Route attribute by putting curly braces around a placeholder.
- - default values can be assigned to route parameters. This default allows the route to match requests when the specified parameter is missing.
- [Route("display/{sid}")]
- `public ContentResult Show(int sid)`
- `{`
- `return Content(sid.ToString());`
- `}`

< > C 88 | localhost:55705/admin/display/77

Inline Constraint for Route

Putting a constraint in the route template in Attribute routing is called an inline • constraint

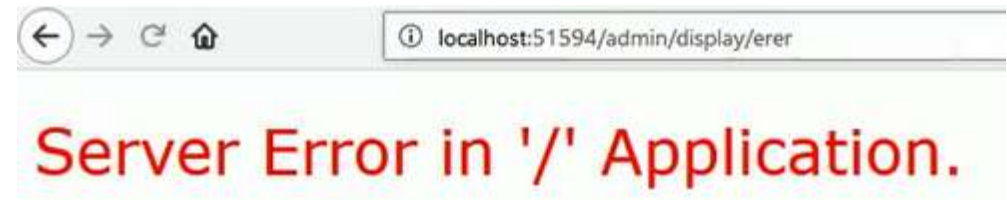
NAME	EXAMPLE USAGE	DESCRIPTION
bool	{n:bool}	A Boolean value
minlength	{n:minlength(2)}	A String value with at least two characters
maxlength	{n:maxlength(2)}	A String value with no more than two characters
length	{n:length(2)}	A String value with exactly two characters
	{n:length(2,4)}	A String value with two, three, or four characters
min	{n:min(1)}	An Int64 value that is greater than or equal to 1

Inline Constraint for Route

NAME	EXAMPLE USAGE	DESCRIPTION
max	<code>{n:max(3)}</code>	An Int64 value that is less than or equal to 3
range	<code>{n:range(1,3)}</code>	The Int64 value 1, 2, or 3
alpha	<code>{n:alpha}</code>	A String value containing only the A–Z and a–z characters
regex	<code>{n:regex (^a+\$)}</code>	A String value containing only one or more 'a' characters (a Regex match for the <code>^a+\$</code> pattern)

Inline Constraint for Route

```
[Route("display/{sid:int}")]  
public ActionResult Show(int sid)  
{  
    return Content(sid.ToString());  
}
```



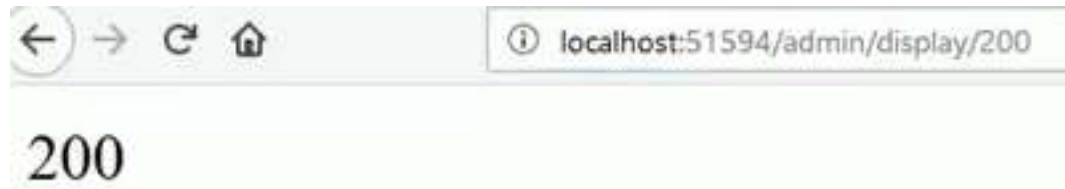
Inline Constraint for Route

```
[Route("display/{sid:range(1,100)}")]  
public ContentResult Show(int sid)  
{  
    return Content(sid.ToString());  
}
```



Inline Constraint for Route

```
[Route("display/{sid:regex(\\d{3})}")]  
public ContentResult Show(int sid)  
{  
    return Content(sid.ToString());  
}
```



Example

```
• public class Student
• {
•     public int Rollno { get; set; }
•     public string Name { get; set; }
•     public string Course { get; set; }
•     public string Address { get; set; }
• }

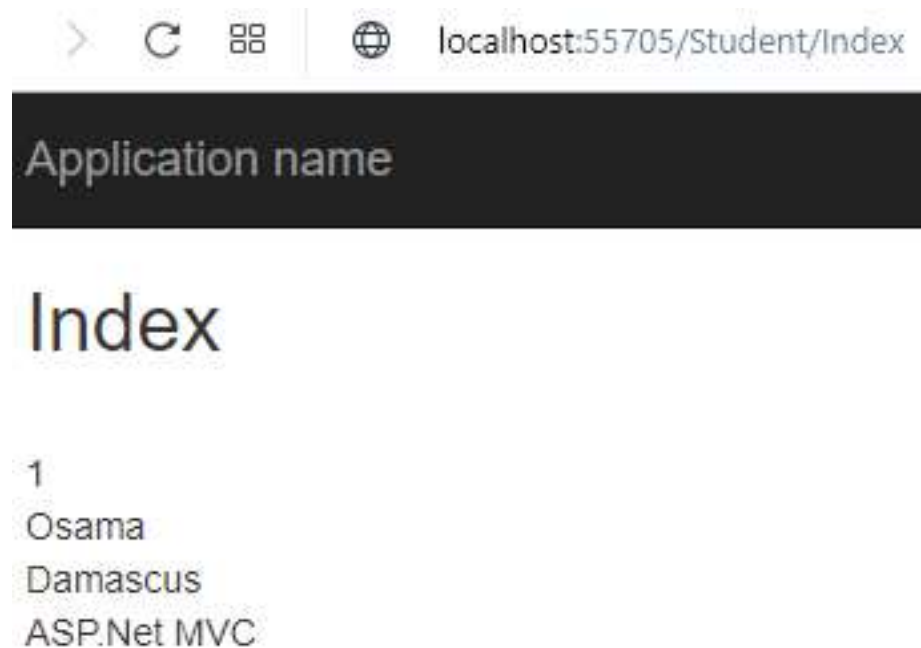
public ActionResult Index()
{
    Student stud = new Student()
    {
        Rollno=1,
        Name="Osama",
        Address="Damascus",
        Course="ASP.Net MVC"
    };
    return View(stud);
}
```


Example

```
@using Ex4.Models  
@model Student
```

```
@{  
    ViewBag.Title = "Index";  
}
```

```
<h2>Index</h2><br />  
@Model.Rollno<br />  
@Model.Name<br />  
@Model.Address<br />  
@Model.Course<br />
```



Example

```
public class StudentController : Controller

public ActionResult List()
{
    List<Student> lst = new List<Student>()
    {
        new Student{Rollno=1,Name="Osama", Address="Damas",Course="MVC"},
        new Student{Rollno=2,Name="Sara", Address="Damas",Course="Core MVC"},
        new Student{Rollno=3,Name="Samer", Address="Damas",Course="PHP"}
    };
    return View(lst);
}
```

Example

```
@using Ex4.Models;
@model IEnumerable<Student>
@{
    ViewBag.Title = "List";
}
<h2>List</h2><hr />
<table>
    <tr>
        <th>Roll No:</th>
        <th>Name:</th>
        <th>Address:</th>
        <th>Course:</th>
    </tr>
    @foreach (Student stud in Model)
    {
        <tr>
            <td>@stud.Rollno</td>
            <td>@stud.Name</td>
            <td>@stud.Address</td>
            <td>@stud.Course</td>
        </tr>
    }
</table>
```

localhost:55705/Student/List

Application name

List

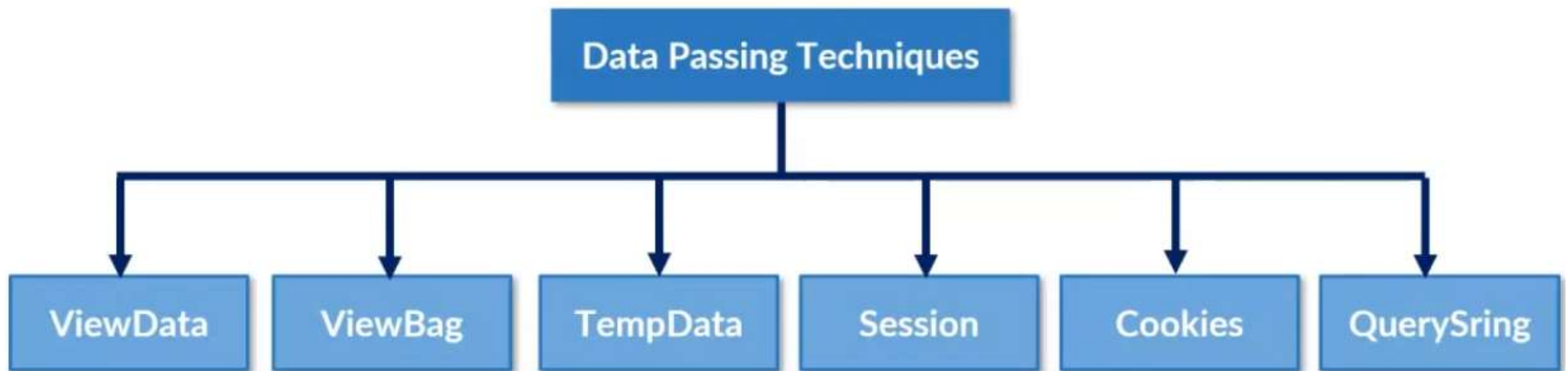
Roll No:	Name:	Address:	Course:
1	OsamaDamas		MVC
2	Sara Damas		Core MVC
2	Samer Damas		PHP

Data Passing Techniques

- The best and popular method to pass the data to the view from the controller is through the help of model classes which are normally known as view class.
- Since in the interface, we normally use the strongly typed views,
- So it is very much easy to access the object or model which need to pass the controller.
- We can perform this operation with the help of the different type of data passing techniques.

Data Passing Techniques

- The following are the object which store value and transfer it from one place to other place:



ViewBag

- The ViewBag is used to transfer temporary data.
- Which is not included in the model from the controller to the view.
- Internally, it is a dynamic type property of the *ControllerBase* class which is the base class of the Controller class.
- It is often used for simple scenarios where a strongly typed model is not required.



```
public class Customer
{
    public int Id { get; set; }
    public string ?Name { get; set; }
    public double Amount { get; set; }
}
```

```
public class TestController : Controller
{
    public static List<Customer> customers = new List<Customer>()
    {
        new Customer(){Id=101,Name="Massa",Amount=12000},
        new Customer(){Id=102,Name="Massa2",Amount=12000},
        new Customer(){Id=103,Name="Massa3",Amount=13000},
        new Customer(){Id=104,Name="Massa4",Amount=14000},
        new Customer(){Id=105,Name="Massa5",Amount=15000}
    };
    public IActionResult Index()
    {
        ViewBag.Message = "Omar suleiman";
        ViewBag.CustomerCount = customers.Count();
        ViewBag.CustomerList = customers;
        return View();
    }
}
```



```
<h1>welcome to the customer:</h1>
```

```
<table class="table">
```

```
  <thead>
```

```
    <tr>
```

```
      <th>Customer Id</th>
```

```
      <th>Customer Name</th>
```

```
      <th>Customer Amount</th>
```

```
    </tr>
```

```
  </thead>
```

```
  <tbody>
```

```
    @foreach (var item in ViewBag.CustomerList)
```

```
    {
```

```
      <tr>
```

```
        <td>@item.Id</td>
```

```
        <td>@item.Name</td>
```

```
        <td>@item.Amount</td>
```

```
      </tr>
```

```
    }
```

```
  </tbody>
```

```
</table>
```

welcome to the customer:

Customer Id	Customer Name	Customer Amount
101	Massa1	12000
102	Massa2	12000
103	Massa3	13000
104	Massa4	14000
105	Massa5	15000

ViewData

- The ViewData is used to pass the data from the controller action method to a view and we can display this data on the view.
- The ViewData is work on the principle of key-value pairs.
- This type of binding is know as loosely binding.
- We can pass any type of data like normal integer, string, even through you can pass objects.
- ViewData only transfers data from controller to view, not vice-versa. It is valid only during the current request.



```
public IActionResult Details()
{
    ViewData["Message"] = "Omar suleiman";
    ViewData["CustomerCount"] = customers.Count();
    ViewData["CustomerList"] = customers;
    return View();
}
```

```
<h1>welcome to the customer:</h1>
```

```
<table class="table">
```

```
  <thead>
```

```
    <tr>
```

```
      <th>Customer Id</th>
```

```
      <th>Customer Name</th>
```

```
      <th>Customer Amount</th>
```

```
    </tr>
```

```
  </thead>
```

```
  <tbody>
```

```
    @foreach (var item in ViewData["CustomerList"] as List<Customer>)  
    {
```

```
      <tr>
```

```
        <td>@item.Id</td>
```

```
        <td>@item.Name</td>
```

```
        <td>@item.Amount</td>
```

```
      </tr>
```

```
    }
```

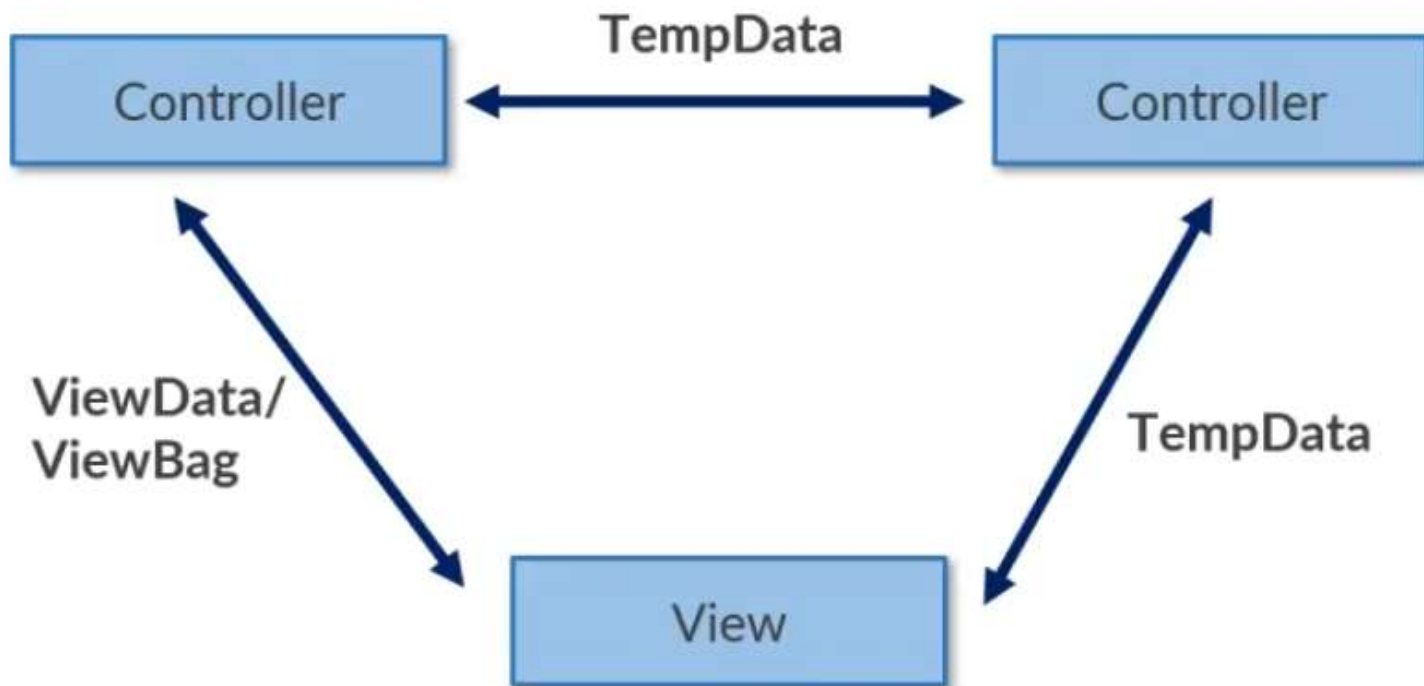
```
  </tbody>
```

```
</table>
```

TempData

- TempData is used to transfer data from the view to the controller, the controller to the view,
- Or from an action method to another action method of the same or a different controller.
- TempData temporarily saves data and deletes it automatically after a value is recovered.
- Unlike ViewBag and ViewData TempData can hold the value for several successive queries.
- TempData can be used in a couple of scenarios like:
 - Passing Data from Controller to View.
 - Transmit data from one activity to another.

TEMPDATA



Uses of TempData

- It is used for storing data between two sequential queries.
- It is similar to a session, which is brief, and uses an internal session to store data.
- We can use the loop to access each key and its value sent to the server.
- To prevent execution time errors, check the Null values.
- Temporary data is mainly used in messages like error messages or validation messages which store a time message.


```
public IActionResult GetTepmData()
{
    TempData["Message"] = "Omar suleiman";
    TempData["CustomerCount"] = customers.Count();
    TempData["CustomerList"] = customers;
    return View();
}
```

```
public IActionResult GetTepmData2()
{
    if (TempData["Message"] == null)
        return RedirectToAction("str");
    TempData["Message"] =
TempData["Message"].ToString();
    return View();
}
```

```
<h1>welcome to the @TempData["Message"]:</h1>
```

```
<p>There are @TempData["CustomerCount"] Customers added to the list.</p>
```

```
<table class="table">
```

```
  <thead>
```

```
    <tr>
```

```
      <th>Customer Id</th>
```

```
      <th>Customer Name</th>
```

```
      <th>Customer Amount</th>
```

```
    </tr>
```

```
  </thead>
```

```
  <tbody>
```

```
    @foreach (var item in TempData["CustomerList"] as List<Customer>)
```

```
    {
```

```
      <tr>
```

```
        <td>@item.Id</td>
```

```
        <td>@item.Name</td>
```

```
        <td>@item.Amount</td>
```

```
      </tr>
```

```
    }
```

```
  </tbody>
```

```
</table>
```

```
<a asp-controller="Test" asp-action="GetTempData2">Go to Method</a>
```

Session

- Session provide a way to store user-specific data across requests.
- A session is a way to persist data between requests for the same user.
- Even if they close their browser or leave the application.
- When user visits a website, server creates a unique session ID, and sends it to client in cookie,
- The client then includes this session ID in subsequent requests,
- Allowing the server to associate the requests with a specific session.
- ASP.NET Core provide a built-in ISession interface that represents a session object.

Session

- In the below example, we are setting a string value “John” in the session with key “UserName”.

```
public IActionResult Index()  
{  
    HttpContext.Session.SetString("UserName", "John");  
    return View();  
}
```

The image shows a Visual Studio window with two panes. The left pane displays the code for `CustomerController.cs` in the `coreMvcFundamentals` project. The code is as follows:

```
1  var builder = WebApplication.CreateBuilder(args);
2
3  // Add services to the container.
4  builder.Services.AddControllersWithViews();
5
6  builder.Services.AddSession(options =>
7  {
8      options.IdleTimeout = TimeSpan.FromSeconds(20);
9      options.Cookie.HttpOnly = true;
10     options.Cookie.IsEssential = true;
11 });
12
13 var app = builder.Build();
14
15 // Configure the HTTP request pipeline.
16 if (!app.Environment.IsDevelopment())
17 {
18     app.UseExceptionHandler("/Home/Error");
19 }
```

The right pane shows the Solution Explorer for the project `coreMvcFundamentals`. The file structure is as follows:

- `coreMvcFundamentals`
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - `CustomerController.cs`
 - `DemoController.cs`
 - `HomeController.cs`
 - Models
 - `Customer.cs`
 - `ErrorViewModel.cs`
 - Views
 - Customer
 - Demo
 - Home
 - Shared
 - `_Layout.cshtml`
 - `_ValidationScriptsPartial.cshtml`
 - `Error.cshtml`
 - `_ViewImports.cshtml`
 - `_ViewStart.cshtml`
 - `appsettings.json`
 - `Program.cs`

Visual Studio interface showing the code for `coreMvcFundamentals.Controllers.DemoController`. The code defines two actions: `Login` and `Success`.

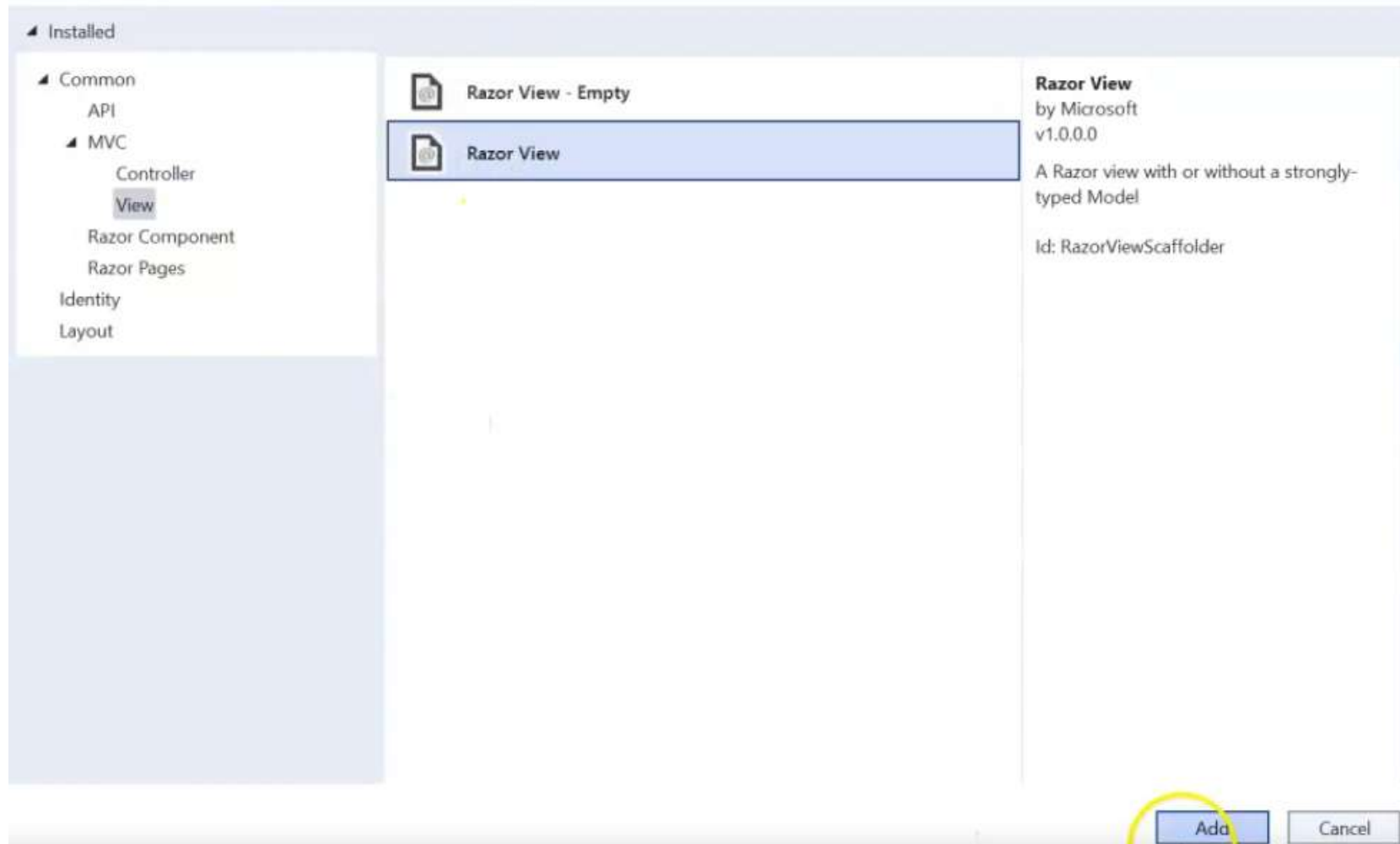
```
42 0 references  
43 public IActionResult Login()  
44 {  
45     HttpContext.Session.SetString("username", "Bhawna");  
46     return RedirectToAction("Success");  
47 }  
48 0 references  
49 public IActionResult Success()  
50 {  
51     ViewBag.Username = HttpContext.Session.GetString("username");  
52     return View();  
53 }  
54 }  
55 }  
56 }
```

The `Success` action is highlighted with a red bracket, indicating it is the current focus. The `Success` action is also highlighted with a red bracket, indicating it is the current focus.

The Solution Explorer on the right shows the project structure for `coreMvcFundamentals`:

- coreMvcFundamentals
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - CustomerController.cs
 - DemoController.cs
 - HomeController.cs
 - Models
 - Customer.cs
 - ErrorViewModel.cs
 - Views
 - Customer
 - _Layout.cshtml
 - _ValidationScriptsPartial.cshtml
 - Error.cshtml
 - _ViewImports.cshtml
 - _ViewStart.cshtml
 - Demo
 - Home
 - Shared
 - appsettings.json
 - Program.cs

Add New Scaffolded Item



Add Razor View

View name	Success
Template	Empty (without model)
Model class	

Options

- ☐ Create as a partial view
- ☐ Reference script libraries
- ☐ Use a layout page

----------------------	----------------------

(Leave empty if it is set in a Razor _viewstart file)

Add	Cancel
------------------------------------	---------------------------------------

Success.cshhtml DemoController.cs Program.cs CustomerController.cs

```
4      }
5
6      <!DOCTYPE html>
7
8      <html>
9      <head>
10         <meta name="viewport" content="width=device-width" />
11         <title>Success</title>
12     </head>
13     <body>
14         <h2>Welcome @ViewBag.Username</h2>
15         <a asp-controller="Demo" asp-action="Logout">Logout</a>
16     </body>
17 </html>
18
```

Solution Explorer

Search Solution Explorer (Ctrl+;)

Solution 'coreExperiencedDevelopersProject'

- coreMvcFundamentals
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - CustomerController.cs
 - DemoController.cs
 - HomeController.cs
 - Models
 - Customer.cs
 - ErrorViewModel.cs
 - Views
 - Customer
 - Demo
 - Home
 - Shared
 - _Layout.cshhtml
 - _ValidationScriptsPartial.cshhtml
 - Error.cshhtml
 - _ViewImports.cshhtml
 - _ViewStart.cshhtml
 - appsettings.json
 - Program.cs

Success.cshhtml DemoController.cs Program.cs CustomerController.cs

coreMvcFundamentals coreMvcFundamentals.Controllers.DemoController Logout()

0 references

```
42 public IActionResult Login()
43 {
44     HttpContext.Session.SetString("username", "Bhawna");
45     return RedirectToAction("Success");
46 }
47
```

0 references

```
48 public IActionResult Success()
49 {
50     ViewBag.Username = HttpContext.Session.GetString("username");
51     return View();
52 }
53
```

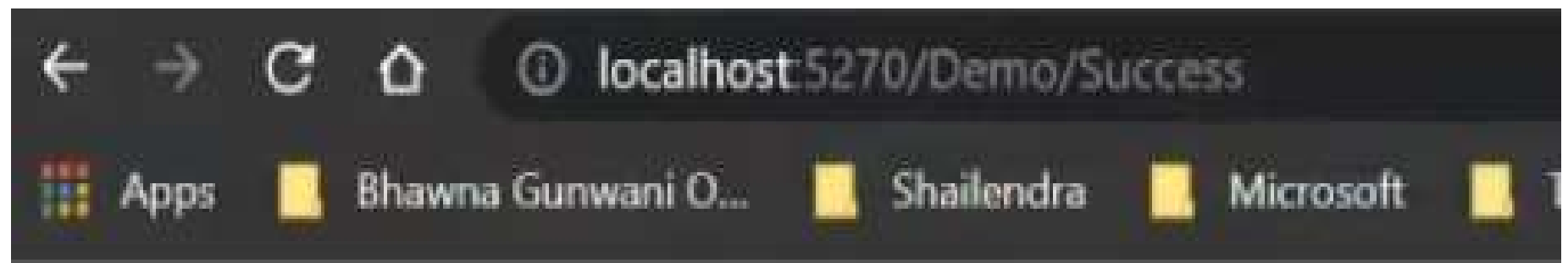
0 references

```
53 public IActionResult Logout()
54 {
55     HttpContext.Session.Remove("Username");
56     return RedirectToAction("Index");
57 }
```

Solution Explorer

Search Solution Explorer (Ctrl+):

- Solution 'coreExperiencedDeve'
- coreMvcFundamentals
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - CustomerController.
 - DemoController.cs
 - HomeController.cs
 - Models
 - Customer.cs
 - ErrorViewModel.cs
 - Views
 - Customer
 - Demo
 - Home
 - Shared
 - _Layout.cshhtml
 - _ValidationScript
 - Error.cshhtml
 - _ViewImports.cshhtml
 - _ViewStart.cshhtml
 - appsettings.json
 - Program.cs



Welcome Bhawna



Query String

- A query string is a collection of characters input to a computer or web browser.
- A Query String is helpful when we want to transfer a value from one page to another.
- When we need to pass content between the HTML pages or aspx WebForms,
- A Query String is very easy to use and the Query String follows a separating character, usually a Question Mark (?).
- It is basically used for identifying data appearing after this separating symbol.
- **Syntax:**

```
Request.QueryString(variable)[(index).count].
```

Visual Studio interface showing the code editor and Solution Explorer.

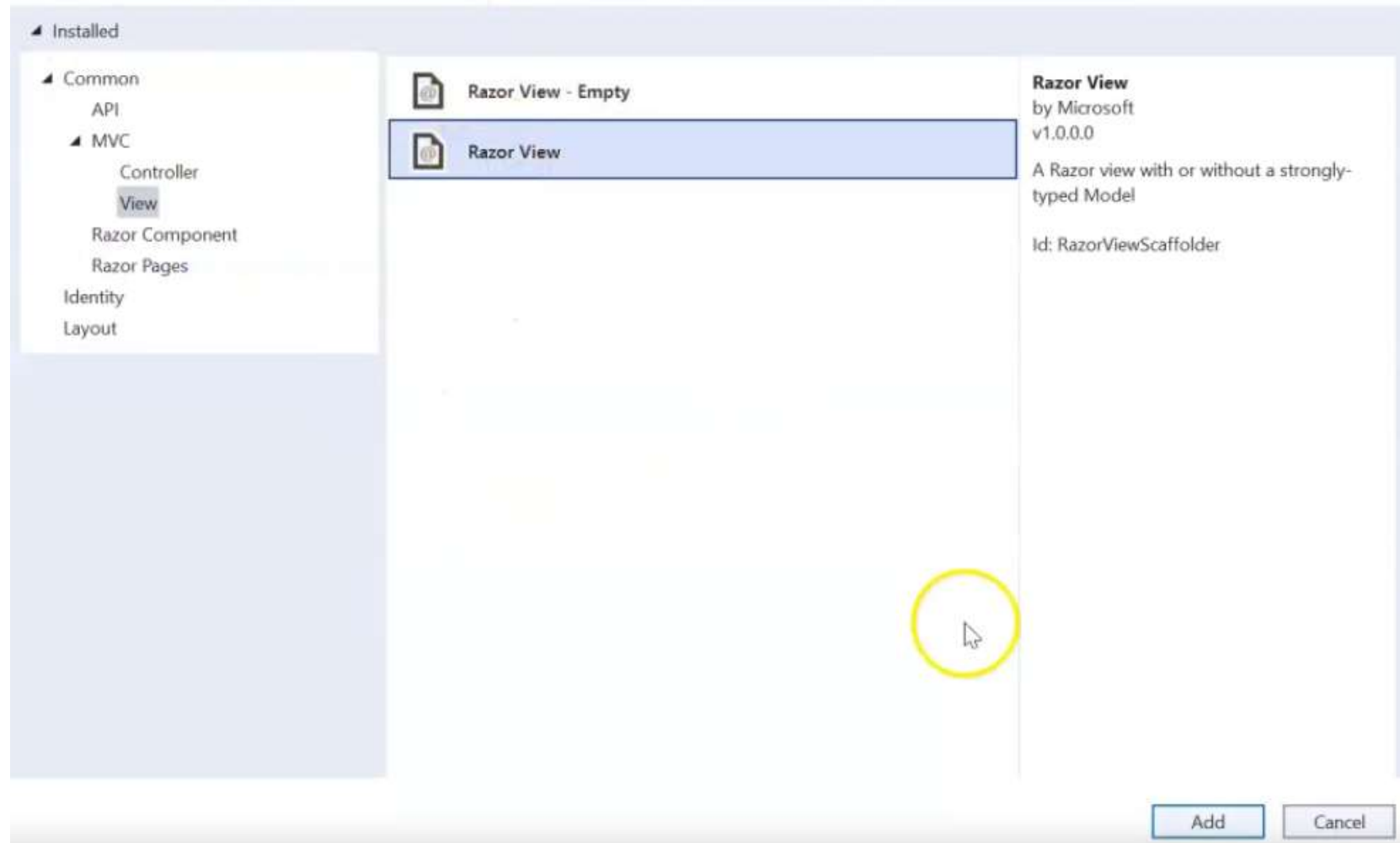
Code Editor:

```
54 {  
55     HttpContext.Session.Remove("Username");  
56     return RedirectToAction("Index");  
57 }  
58  
59 0 references  
60 public IActionResult QueryTest()  
61 {  
62     string name = "King Kochhar";  
63     if (!String.IsNullOrEmpty(HttpContext.Request.Query["name"]))  
64         name = HttpContext.Request.Query["name"];  
65     ViewBag.Name = name;  
66     return View();  
67 }  
68 }  
69 }  
70
```

Solution Explorer:

- Solution 'coreExperiencedDevelopersProjects'
 - coreMvcFundamentals
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - CustomerController.cs
 - DemoController.cs
 - HomeController.cs
 - Models
 - Customer.cs
 - ErrorViewModel.cs
 - Views
 - Customer
 - Demo
 - Home
 - Shared
 - _Layout.cshtml
 - _ValidationScriptsPartial.cshtml
 - Error.cshtml
 - _ViewImports.cshtml
 - _ViewStart.cshtml
 - appsettings.json

Add New Scaffolded Item



Add Razor View

View name	QueryTest
Template	Empty (without model)
Model class	

Options

- ☐ Create as a partial view
- ☐ Reference script libraries
- ☐ Use a layout page

----------------------	----------------------

(Leave empty if it is set in a Razor _viewstart file)

☒ Add	Cancel
--------------------------------------	--------

QueryTest.cshtml

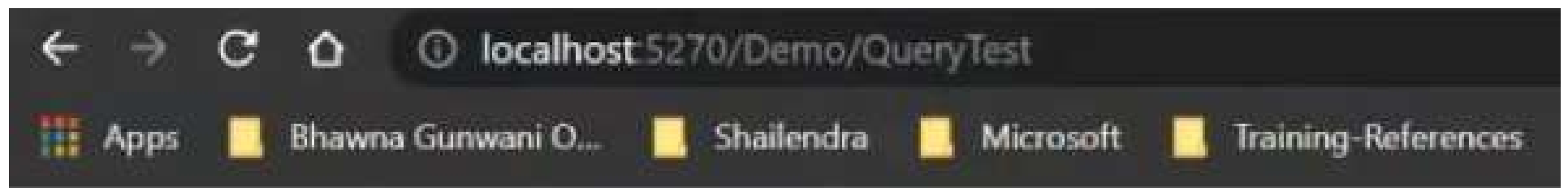
Success.cshtml

DemoController.cs

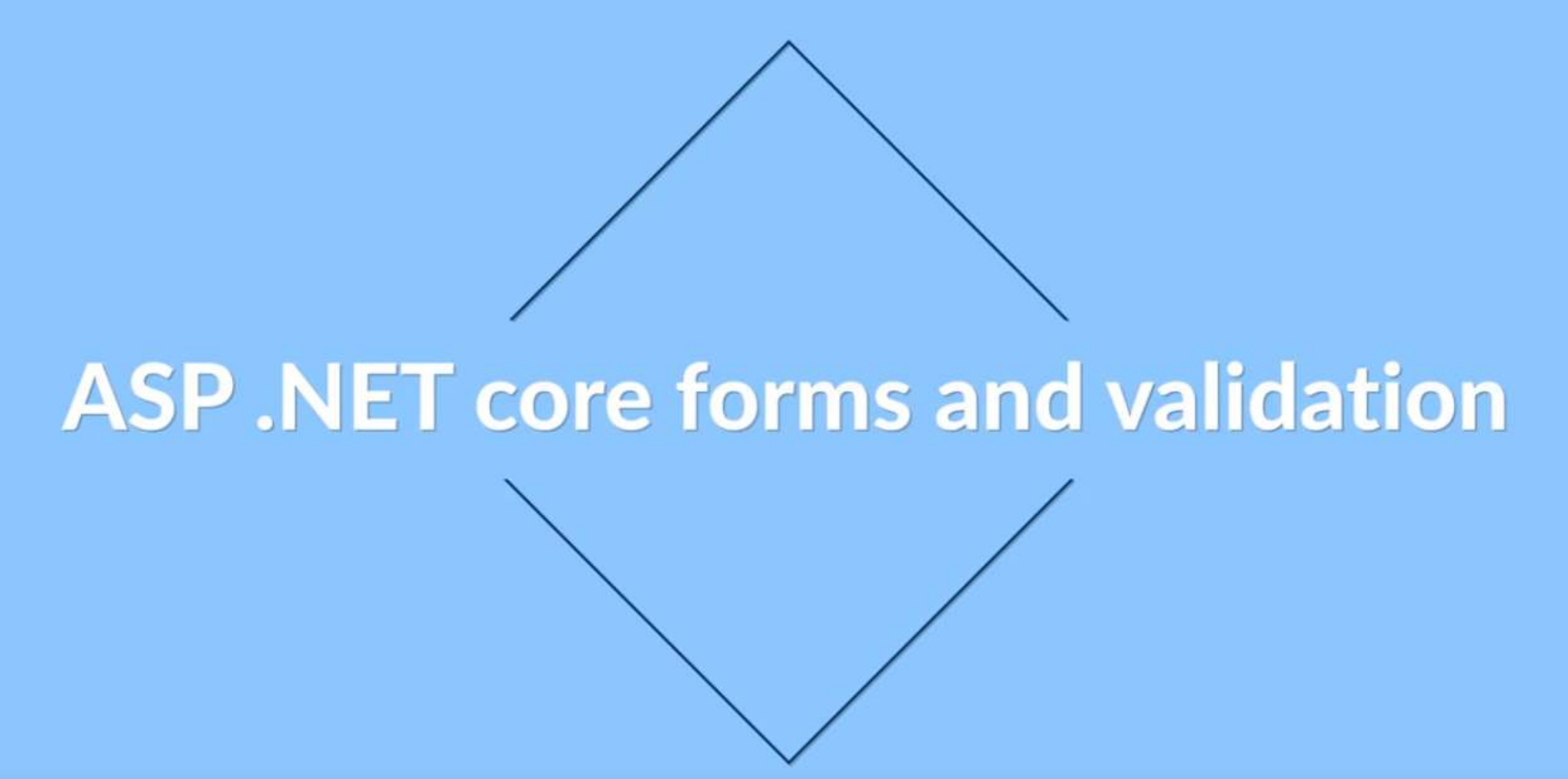
Program.cs

CustomerController.cs

```
1
2  @{
3      Layout = null;
4  }
5
6  <!DOCTYPE html>
7
8  <html>
9  <head>
10     <meta name="viewport" content="width=device-width" />
11     <title>QueryTest</title>
12 </head>
13 <body> I
14     <h2>@ViewBag.Name</h2>
15 </body>
16 </html>
17
```



King Kochhar



ASP.NET core forms and validation

- Types of forms in ASP.NET Core
- How to handle them using model binding
- Form validation
- Server side & client side validation
- How to implement custom validation.

ASP.NET Core Forms

- Forms are a key part of building web applications using the ASP.NET Core framework.
- Forms allow users to submit data to a server.
- Which can then be used to create, update, or delete records, or to perform some other action.
- They can be created using HTML Helpers, which are methods that generate HTML code that can be used to display and interact with forms.
- When a user submits a form, the data is typically sent to the server a POST request.
- The server then receives the data and can use it to perform the desired action, such as updating a database record.

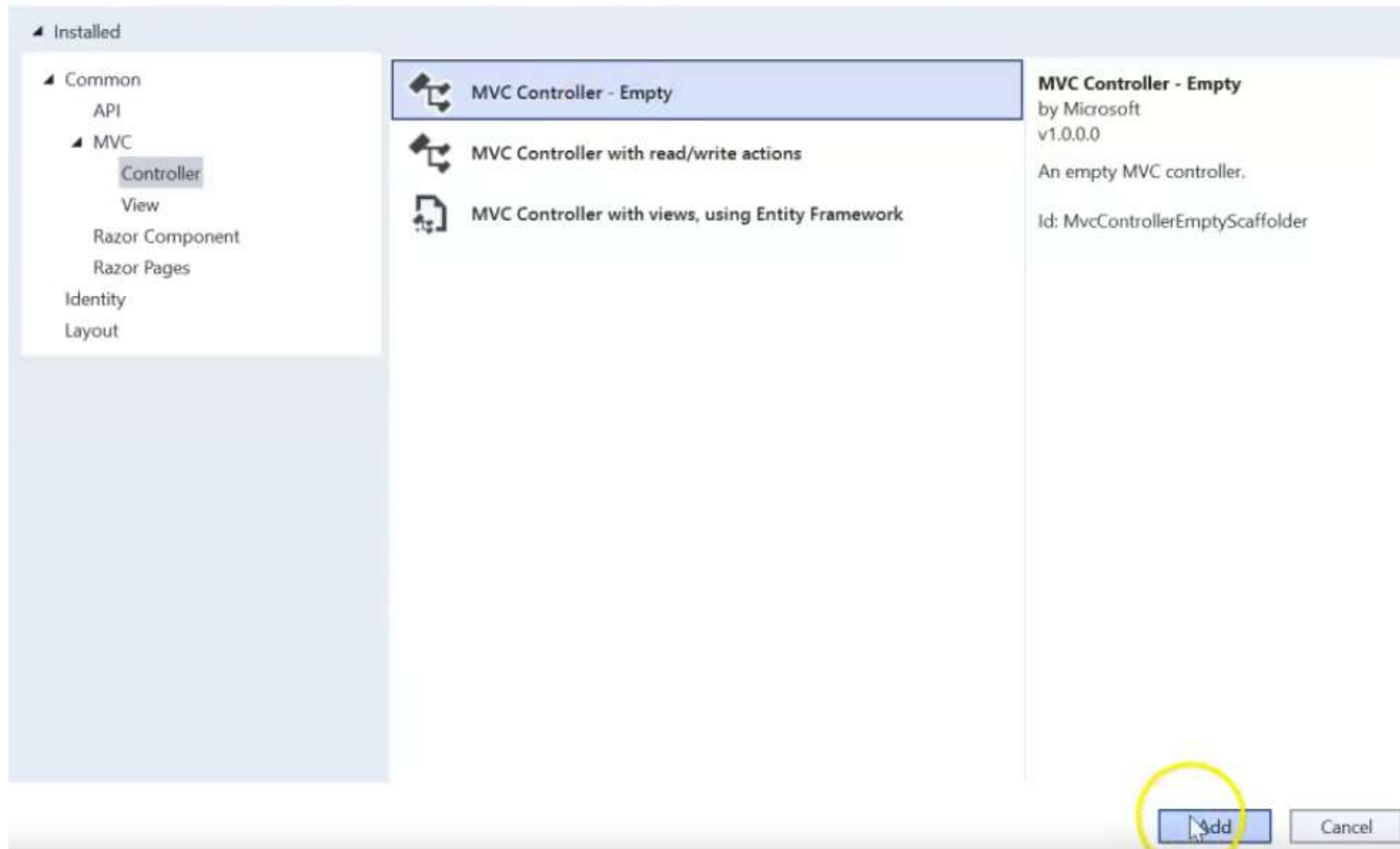
Weakly-Typed Forms

- it refers to a way of handling form data where data is not explicitly bound to specific data model.
- Instead, the form data is accessed as a collection of key-value pairs,
- Where the keys represent the names of the form fields and the values represent the user input.
- The developer can access using the Request object, which provides access to the form collection.
- This allows the developer to read and manipulate the form data directly, without the need to create a specific data model.

Strongly Forms

- it refers to a way of handling form data where data is explicitly bound to specific data model.
- This allows developers to define a data model that corresponds to the structure of the form data,
- And to use model binding to automatically map the form data to the data model.
- The developer can create a data model using a class or a structure that defines the properties and data types of the form fields.
- The form data is then automatically mapped to properties of the data model using model binding,
- Which provides compile-time validation and reduces the risk of runtime errors.

Add New Scaffolded Item



AccountController.cs

coreFormsAndValidations

coreFormsAndValidations.Controllers.AccountC

Index()

```
1 using Microsoft.AspNetCore.Mvc;
2
3 namespace coreFormsAndValidations.Controllers
4 {
5     public class AccountController : Controller
6     {
7         public IActionResult Index()
8         {
9             return View();
10        }
11    }
12 }
13
```

Solution Explorer

Search Solution Explorer (Ctrl+J)

Solution 'coreExperiencedDevelopersProjects' (2 of 2 p

- coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - C# AccountController.cs
 - C# HomeController.cs
 - Models
 - Views
 - appsettings.json
 - C# Program.cs
- coreMvcFundamentals

AccountController.cs* X

coreFormsAndValidations coreFormsAndValidations.Controllers.AccountC WeeklyTypedLogin()

```
1 using Microsoft.AspNetCore.Mvc;
2
3 namespace coreFormsAndValidations.Controllers
4 {
5     public class AccountController : Controller
6     {
7         public IActionResult WeeklyTypedLogin()
8         {
9             return View();
10        }
11    }
12 }
```

0 references

I

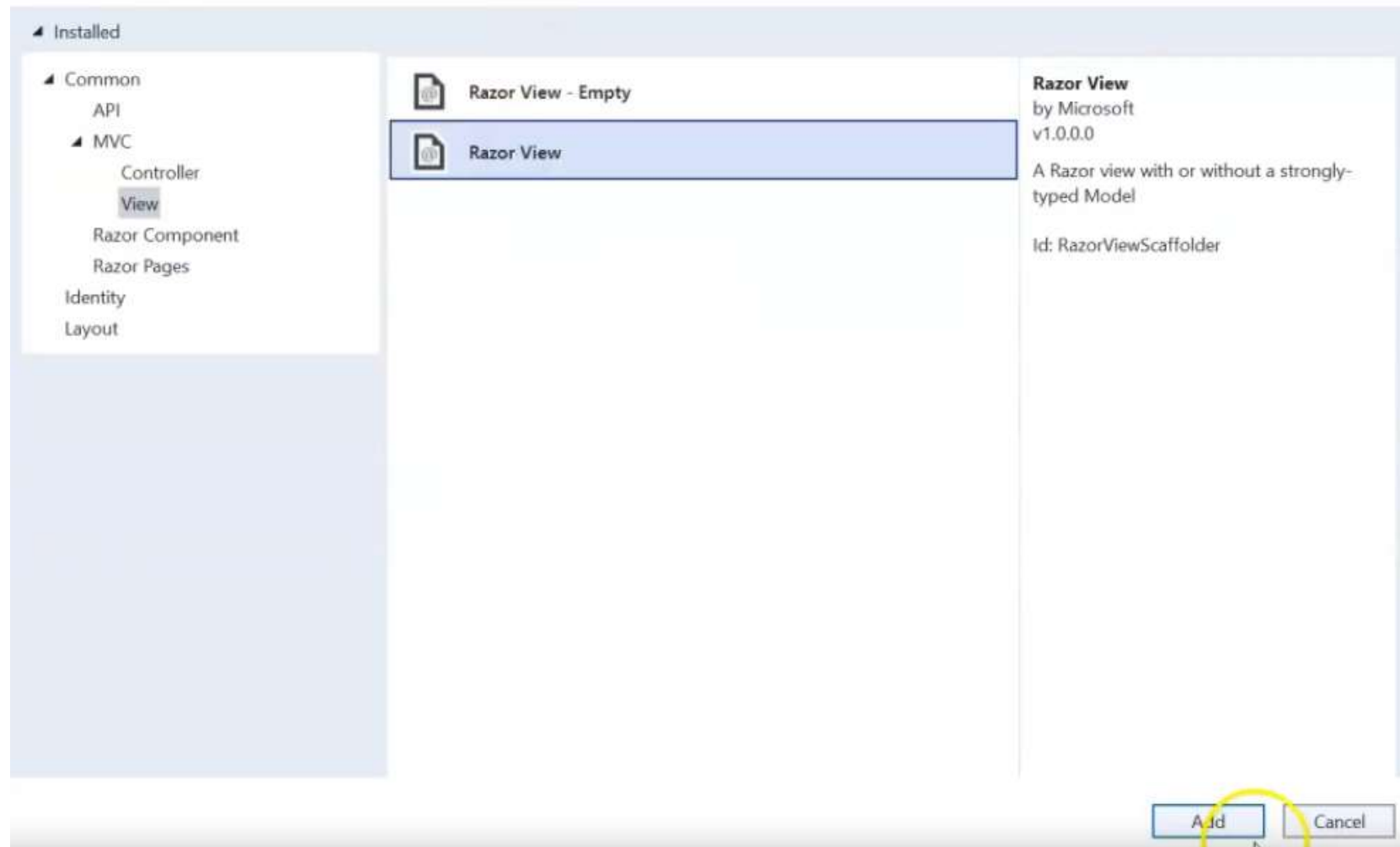
Solution Explorer

Search Solution Explorer (Ctrl+.)

Solution 'coreExperiencedDevelopersProjects' (2 of 2 p

- coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - C# AccountController.cs
 - C# HomeController.cs
 - Models
 - Views
 - appsettings.json
 - C# Program.cs
- coreMvcFundamentals

Add New Scaffolded Item



Add Razor View

View name

Template

Model class

Options

- ☐ Create as a partial view
- ☒ Reference script libraries
- ☐ Use a layout page

(Leave empty if it is set in a Razor _viewstart file)

```
WeeklyTypedLogin.cshtml | AccountController.cs
7
8 <html>
9 <head>
10   <meta name="viewport" content="width=device-width" />
11   <title>WeeklyTypedLogin</title>
12 </head>
13 <body>
14   @using(Html.BeginForm())
15   {
16     <div>
17       Username: @Html.TextBox("Username")
18     </div>
19     <div>
20       Password: @Html.TextBox("Password")
21     </div>
22     <div>
23       <input type="submit" value="Login" />
```

```
WeeklyTypedLogin.cshtml | AccountController.cs
10  <meta name="viewport" content="width=device-width" />
11  <title>WeeklyTypedLogin</title>
12  </head>
13  <body>
14  @using(Html.BeginForm("LoginPost", "Account", FormMethod.Post))
15  {
16      <div>
17          Username: @Html.TextBox("Username")
18      </div>
19      <div>
20          Password: @Html.TextBox("Password")
21      </div>
22      <div>
23          <input type="submit" value="Login" />
24      </div>
25  }
26  </body>
```

Solution Explorer

- Solution 'coreExperiencedDevelopersProjects' (2 of ...)
- coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - C# AccountController.cs
 - C# HomeController.cs
 - Models
 - Views
 - appsettings.json
 - C# Program.cs
- coreMvcFundamentals

WeeklyTypedLogin.cshhtml | AccountController.cs | LoginPost()

coreFormsAndValidations | coreFormsAndValidations.Controllers.AccountC | LoginPost()

```
4 {
5     0 references
6     public class AccountController : Controller
7     {
8         0 references
9         public IActionResult WeeklyTypedLogin()
10        {
11            return View();
12        }
13        [HttpPost]
14        0 references
15        public IActionResult LoginPost()
16        {
17            return View();
18        }
19    }
20 }
```

Solution Explorer

Search Solution Explorer (Ctrl+.)

Solution 'coreExperiencedDevelopersProjects' (2 of 2)

- coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - AccountController.cs
 - HomeController.cs
 - Models
 - Views
 - appsettings.json
 - Program.cs
- coreMvcFundamentals

WeeklyTypedLogin.cshhtml AccountController.cs

coreFormsAndValidations coreFormsAndValidations.Controllers.AccountCr LoginPost(string username, string password)

```
5 public class AccountController : Controller
6 {
7     0 references
8     public IActionResult WeeklyTypedLogin()
9     {
10         return View();
11     }
12     [HttpPost]
13     0 references
14     public IActionResult LoginPost(string username, string password)
15     {
16         return View();
17     }
18 }
```

Solution Explorer

Search Solution Explorer (Ctrl+;)

Solution 'coreExperiencedDevelopersProjects' (2 of 2 p

- coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - C# AccountController.cs
 - C# HomeController.cs
 - Models
 - Views
 - appsettings.json
 - C# Program.cs
 - coreMvcFundamentals

Add Razor View

View name	LoginPost
Template	Empty (without model)
Model class	

Options

- ☐ Create as a partial view
- ☐ Reference script libraries
- ☐ Use a layout page

	...
----------------------	------------------------------------

(Leave empty if it is set in a Razor _viewstart file)

Add	Cancel
------------------------------------	---------------------------------------

Visual Studio interface showing the **AccountController.cs** file in the **coreFormsAndValidations** project.

Code in AccountController.cs:

```
5 public class AccountController : Controller
6 {
7     0 references
8     public IActionResult WeeklyTypedLogin()
9     {
10         return View();
11     }
12     [HttpPost]
13     0 references
14     public IActionResult LoginPost(string username, string password)
15     {
16         ViewBag.Username = username;
17         ViewBag.Password = password;
18         return View();
19     }
20 }
```

Solution Explorer (Right Panel):

- Solution 'coreExperiencedDevelopersProjects' (2 of ...)
 - coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - AccountController.cs
 - HomeController.cs
 - Models
 - Views
 - appsettings.json
 - Program.cs
 - coreMvcFundamentals

Visual Studio interface showing the development of a web application. The main editor displays the `LoginPost.cshtml` file, which is a Razor view. The code includes a layout declaration, a DOCTYPE declaration, and an HTML document structure with a head section containing a viewport meta tag and a title, and a body section containing a welcome message.

```
1 @{\n2     Layout = null;\n3\n4\n5\n6 <!DOCTYPE html>\n7\n8 <html>\n9 <head>\n10     <meta name="viewport" content="width=device-width" />\n11     <title>LoginPost</title>\n12 </head>\n13 <body>\n14     Welcome @ViewBag.Username, Your Password is @ViewBag.Password.\n15 </body>\n16 </html>
```

The Solution Explorer on the right shows the project structure for 'coreExperiencedDevelopersProjects' (2 of 2 projects). The selected project is 'coreFormsAndValidations', which contains the following files and folders:

- Connected Services
- Dependencies
- Properties
- wwwroot
- Controllers
 - AccountController.cs
 - HomeController.cs
- Models
- Views
- appsettings.json
- Program.cs
- coreMvcFundamentals

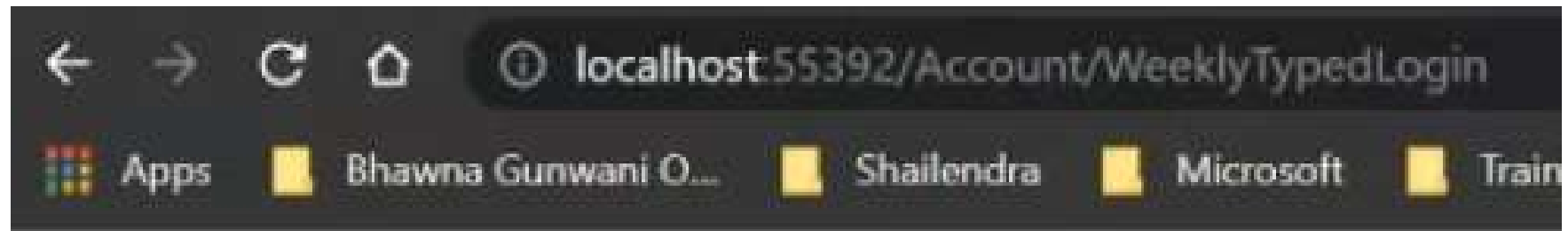
Visual Studio interface showing the code for `WeeklyTypedLogin.cshtml` and the `Solution Explorer`.

Code Editor (WeeklyTypedLogin.cshtml):

```
10 <meta name="viewport" content="width=device-width" />
11 <title>WeeklyTypedLogin</title>
12 </head>
13 <body>
14     @using(Html.BeginForm("LoginPost", "Account", FormMethod.Post))
15     {
16         <div>
17             Username: @Html.TextBox("Username")
18         </div>
19         <div>
20             Password: @Html.Password("Password")
21         </div>
22         <div>
23             <input type="submit" value="Login" />
24         </div>
25     }
26 </body>
```

Solution Explorer:

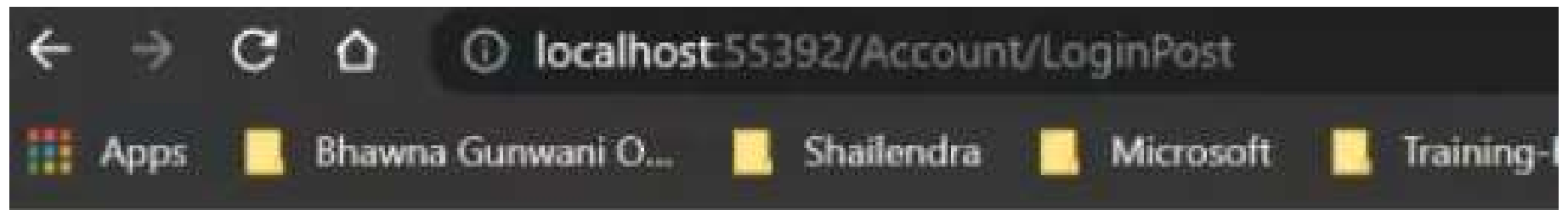
- Solution 'coreExperiencedDevelopersProjects' (2 of 2 projects)
 - coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - C# AccountController.cs
 - C# HomeController.cs
 - Models
 - Views
 - appsettings.json
 - C# Program.cs
 - coreMvcFundamentals



Username:

Password:

Login



Welcome king, Your Password is 123456.

DEMO: STRONGLY TYPED FORMS



Add New Item - coreFormsAndValidations

Installed

Sort by: Default

Search (Ctrl+E)

C#

General

ASP.NET Core

Code

Data

General

Web

CSharp

Online

	Class	C#	Type: C# An empty class declaration
	Interface	C#	
	Code File	C#	

Name:

LoginViewModel.cs

Add

Cancel

Visual Studio interface showing the code editor and Solution Explorer.

Code Editor (LoginViewModel.cs):

```
1 namespace coreFormsAndValidations.Models
2 {
3     0 references
4     public class LoginViewModel
5     {
6         0 references
7         public string? Username { get; set; }
8         0 references
9         public string? Password { get; set; }
10    }
11 }
```

Solution Explorer:

- Solution 'coreExperiencedDevelopersProjects' (2 of 2 projects)
 - coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - AccountController.cs
 - HomeController.cs
 - Models**
 - ErrorViewModel.cs
 - LoginViewModel.cs
 - Views
 - appsettings.json
 - Program.cs
 - coreMvcFundamentals

Visual Studio interface showing the **AccountController.cs** file in the **coreFormsAndValidations.Controllers.AccountC** namespace. The file contains two action methods: **LoginPost** and **StronglyTypedLogin()**.

```
12 public IActionResult LoginPost(string username, string password)
13 {
14     ViewBag.Username = username;
15     ViewBag.Password = password;
16     return View();
17 }
18 public IActionResult StronglyTypedLogin()
19 {
20     return View();
21 }
22 }
23 }
```

The **StronglyTypedLogin()** method is highlighted with a red underline. The **return View();** statement in the **StronglyTypedLogin()** method is highlighted with a red bracket. The **return View();** statement in the **LoginPost** method is highlighted with a green bracket.

The **Solution Explorer** on the right shows the project structure for **coreFormsAndValidations**, including **Controllers** (AccountController.cs, HomeController.cs) and **Models** (ErrorViewModel.cs, LoginViewModel.cs).

Add Razor View

View name	StronglyTypedLogin
Template	Empty (without model)
Model class	

Options

- ☐ Create as a partial view
- ☐ Reference script libraries
- ☐ Use a layout page

----------------------	----------------------

(Leave empty if it is set in a Razor _viewstart file)

Add	Cancel
------------------------------------	---------------------------------------

StronglyTypedLogin.cshtml | LoginViewModel.cs | LoginPost.cshtml | WeeklyTypedLogin.cshtml

```
2  @model coreFormsAndValidations.Models.LoginViewModel
3
4  @{
5      Layout = null;
6  }
7
8  <!DOCTYPE html>
9
10 <html>
11 <head>
12     <meta name="viewport" content="width=device-width" />
13     <title>StronglyTypedLogin</title>
14 </head>
15 <body>
16 </body>
17 </html>
```

Solution Explorer

Search Solution Explorer (Ctrl+;)

- Solution 'coreExperiencedDevelopersProje'
 - coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - AccountController.cs
 - HomeController.cs
 - Models
 - ErrorViewModel.cs
 - LoginViewModel.cs
 - Views
 - appsettings.json
 - Program.cs
 - coreMvcFundamentals

StronglyTypedLogin.cshtml | LoginViewModel.cs | LoginPost.cshtml | WeeklyTypedLogin.cshtml

```
14 </head>
15 <body>
16     @using(Html.BeginForm("LoginSuccess", "Account", FormMethod.Post))
17     {
18         <div>
19             Username: Html.TextBoxFor( m => m.Username)
20         </div>
21         <div>
22             Password: Html.PasswordFor( m => m.Password)
23         </div>
24         <div>
25             <input type="submit" value="Login">
26         </div>
27     }
28 </body>
29 </html>
```

Solution Explorer

Search Solution Explorer (Ctrl+;)

- Solution 'coreExperiencedDevelopersP'
 - coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - AccountController.cs
 - HomeController.cs
 - Models
 - ErrorViewModel.cs
 - LoginViewModel.cs
 - Views
 - appsettings.json
 - Program.cs
 - coreMvcFundamentals

StronglyTypedLogin.cshtml | LoginViewModel.cs | LoginPost.cshtml | WeeklyTypedLogin.cshtml | AccountController.cs* | Solution Explorer

coreFormsAndValidations | coreFormsAndValidations.Controllers.AccountContr | LoginSuccess(LoginViewModel login)

```
13 public IActionResult LoginPost(string username, string password)
14 {
15     ViewBag.Username = username;
16     ViewBag.Password = password;
17     return View();
18 }
19 public IActionResult StronglyTypedLogin()
20 {
21     return View();
22 }
23 [HttpPost]
24 public IActionResult LoginSuccess(LoginViewModel login)
25 {
26     ViewBag.Username = login.Username;
27     ViewBag.Password = login.Password;
28     return View();
29 }
30 }
31 }
```

0 references

0 references

0 references

Solution 'coreExperiencedDevelopersProjects'

- coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - AccountController.cs
 - HomeController.cs
 - Models
 - ErrorViewModel.cs
 - LoginViewModel.cs
 - Views
 - appsettings.json
 - C# Program.cs
 - coreMvcFundamentals

Add Razor View

View name

Template

Model class

Options

- ☐ Create as a partial view
- ☐ Reference script libraries
- ☐ Use a layout page

(Leave empty if it is set in a Razor _viewstart file)



LoginSuccess.cshtmlStronglyTypedLogin.cshtmlLoginViewModel.csLoginPost.cshtmlWeeklyTypedLogin.cshtml

1234567891011121314151617

@{
Layout = null;
}

<!DOCTYPE html>

<html>
<head>
 <meta name="viewport" content="width=device-width" />
 <title>LoginSuccess</title>
</head>
<body>
 Welcome @ViewBag.Username, Your Password is @ViewBag.Password.
</body>
</html>

Solution Explorer

Search Solution Explorer (Ctrl+;)

Solution 'coreExperiencedDevelopersProjects' (

coreFormsAndValidations

Connected Services

Dependencies

Properties

wwwroot

Controllers

AccountController.cs

HomeController.cs

Models

ErrorViewModel.cs

LoginViewModel.cs

Views

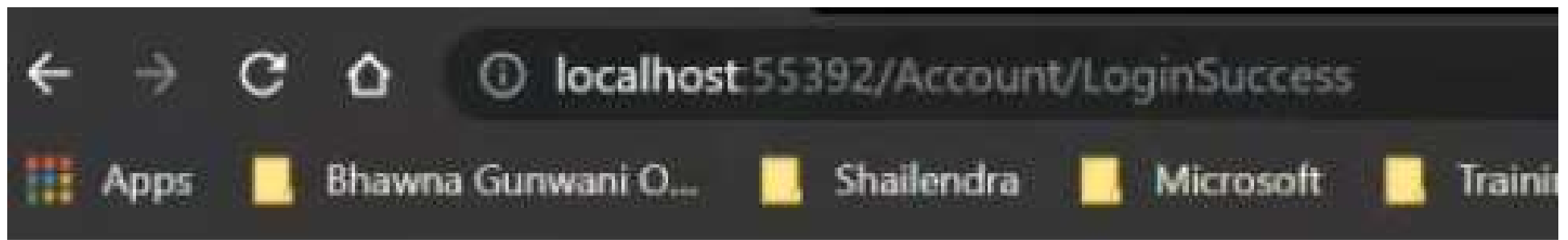
appsettings.json

Program.cs

coreMvcFundamentals



Username: `Html.TextBoxFor( m => m.Username)`
Password: `Html.PasswordFor( m => m.Password)`



Welcome king, Your Password is 123456.

Model Binding

- This is a mechanism that extracts the data from an HTTP request and provides them to the controller action method parameters.
- The action method parameters may be simple types:
 - Like integers, strings, etc., or complex types such as Student, Order, Product, etc..
- When a client makes a request to a controller action method.
- ASP.NET Core automatically maps the data in the request to the action method's parameters using model binding.
- Models binding is a powerful feature that can simplify your code and reduce boilerplate.

How Model Binding Works?

- For example, let's say you have an action method that takes a *Person* Object as a parameter:

```
public IActionResult UpdatePerson(Person person)
{
    // update the person
    return View(person);
}
```

- When the client submits a form that includes data for a Person object.

```
public IActionResult UserDetails()  
{  
    var user = new LoginViewModel() { Username = "Bhawna", Password = "123456" };  
    return View(user);  
}
```

Add Razor View

View name

Template

Model class

Options

- ☐ Create as a partial view
- ☐ Reference script libraries
- ☐ Use a layout page

(Leave empty if it is set in a Razor _viewstart file)

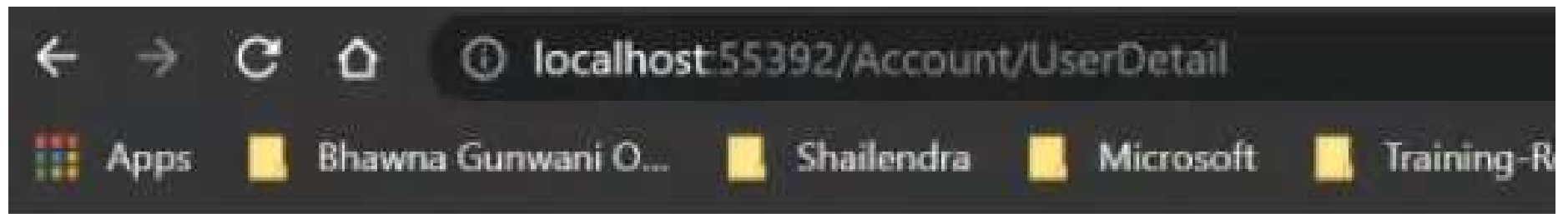
Visual Studio interface showing the code editor and Solution Explorer.

Code Editor (UserDetail.cshtml):

```
1
2 @model coreFormsAndValidations.Models.LoginViewModel
3
4 @{
5     Layout = null;
6 }
7
8 <!DOCTYPE html>
9
10 <html>
11 <head>
12     <meta name="viewport" content="width=device-width" />
13     <title>UserDetail</title>
14 </head>
15 <body>
16 </body>
17 </html>
18
```

Solution Explorer:

- Solution 'coreExperiencedDevelopersProjects' (...)
 - coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - C# AccountController.cs
 - C# HomeController.cs
 - Models
 - C# ErrorViewModel.cs
 - C# LoginViewModel.cs
 - Views
 - appsettings.json
 - C# Program.cs
 - coreMvcFundamentals



User Details

Username: Bhawna

Username: 123456


```
UserDetail.cshtml  AccountController.cs
1
2  @model coreFormsAndValidations.Models.LoginViewModel
3
4  @{
5      Layout = null;
6  }
7
8  <!DOCTYPE html>
9
10 <html>
11 <head>
12     <meta name="viewport" content="width=device-width" />
13     <title>UserDetail</title>
14 </head>
15 <body>
16     <h2>User Details</h2>
17     <p>Username: @Model.Username</p>
18     <p>Password: @Model.Password</p>
19 </body>
20 </html>
21
```

Visual Studio interface showing the **AccountController.cs** file in the **coreFormsAndValidations** project.

Code in AccountController.cs:

```
30  
31 public IActionResult UserDetails()  
32 {  
33     var user = new LoginViewModel() { Username = "Bhawna", Password = "123456" };  
34     return View(user);  
35 }  
36 public IActionResult UserList()  
37 {  
38     var users = new List<LoginViewModel>()  
39     {  
40         new LoginViewModel() { Username = "Bhawna", Password = "123456" },  
41         new LoginViewModel() { Username = "Hitesh", Password = "123456" },  
42         new LoginViewModel() { Username = "Kaashvi", Password = "123456" }  
43     };  
44     return View(users);  
45 }  
46 }  
47 }
```

Solution Explorer (Right Panel):

- Solution 'coreExperiencedDevelopersProjects'
 - coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - AccountController.cs
 - HomeController.cs
 - Models
 - ErrorViewModel.cs
 - LoginViewModel.cs
 - Views
 - appsettings.json
 - C# Program.cs
 - coreMvcFundamentals

Add Razor View

View name

Template

Model class

Options

- ☐ Create as a partial view
- ☐ Reference script libraries
- ☐ Use a layout page

(Leave empty if it is set in a Razor _viewstart file)

Add

Cancel

```
UserList.cshtml x UserDetail.cshtml AccountController.cs
1 @model IList<coreFormsAndValidations.Models.LoginViewModel>
2
3 @{
4     Layout = null;
5 }
6
7 <!DOCTYPE html>
8
9 <html>
10 <head>
11     <meta name="viewport" content="width=device-width" />
12     <title>UserList</title>
13 </head>
14 <body>
15     <ul>
16         @foreach(var item in Model)
17         {
18             <li>@item.Username - @item.Password</li>
19         }
20     </ul>
21
22 </body>
23 </html>
24
```

DEMO: HANDLING FORMS POST

StronglyTypedLogin.cshtmlAccountController.cs

coreFormsAndValidationscoreFormsAndValidations.Controllers.AccountControLoginSuccess(LoginViewModel login)

0 references

19 public IActionResult StronglyTypedLogin()
20 {
21 return View();
22 }
23 [HttpPost]
24 public IActionResult LoginSuccess(LoginViewModel login)
25 {
26 if(login.Username != null && login.Password != null)
27 {
28 if(login.Username.Equals("admin") && login.Password.Equals("admin"))
29 {
30 ViewBag.Message = "You are successfully logged in.";
31 return View();
32 }
33 }
34 ViewBag.Message = "Invalid Credentials";
35 return View();
36 }
37

Visual Studio interface showing a C# project named 'coreExperiencedDevelopersProjects'.

The main editor displays the 'AccountController.cs' file, which is highlighted with a yellow circle in the tab bar. The code is as follows:

```
1
2  @{
3      Layout = null;
4  }
5
6  <!DOCTYPE html>
7
8  <html>
9  <head>
10     <meta name="viewport" content="width=device-width" />
11     <title>LoginSuccess</title>
12 </head>
13 <body>
14     @*Welcome @ViewBag.Username, Your Password is @ViewBag.Password.*@
15     @ViewBag.Message
16 </body>
17 </html>
18
```

The Solution Explorer on the right shows the project structure:

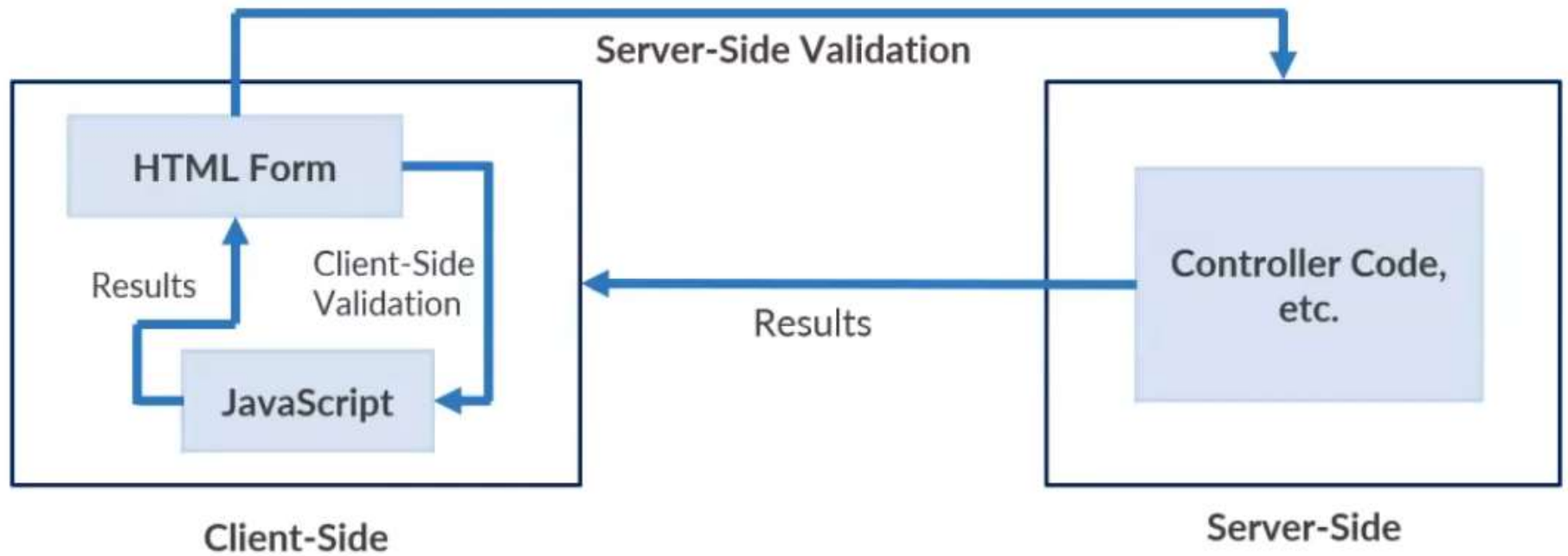
- Solution 'coreExperiencedDevelopersProjects'
 - coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - AccountController.cs
 - HomeController.cs
 - Models
 - ErrorViewModel.cs
 - LoginViewModel.cs
 - Views
 - appsettings.json
 - Program.cs
 - coreMvcFundamentals

FORMS VALIDATIONS - SERVER SIDE AND CLIENT SIDE

Form validations

- Form validation is the process of ensuring that user input data submitted through a web form is valid, and meets certain criteria or rules.
- It helps to ensure that user input is accurate and consistent with the expected format.
- There are different types of form validations:
 - Server side form validation
 - Client side form validation
 - Custom form validation

FORM VALIDATIONS



Add New Item - coreFormsAndValidations

? X

Installed

Sort by: Default

Search (Ctrl+E)

C#

General

ASP.NET Core

Code

Data

General

Web

CSharp

Online

 Class

 Interface

 Code File

C#

C#

C#

Type: C#

An empty class declaration

Name: Account.cs

Add Cancel

Account.cs X

coreFormsAndValidations coreFormsAndValidations.Models.Account Website

```
1 namespace coreFormsAndValidations.Models
2 {
3     0 references
4     public class Account
5     {
6         0 references
7         public string? Username { get; set; }
8         0 references
9         public string? Password { get; set; }
10        0 references
11        public int Age { get; set; }
12        0 references
13        public string? Email { get; set; }
14        0 references
15        public string? Website { get; set; }
16    }
17 }
```

Solution Explorer

Search Solution Explorer (Ctrl+;)

Solution 'coreExperiencedDevelopersPr'

- coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - AccountController.cs
 - HomeController.cs
 - Models
 - Account.cs
 - ErrorViewModel.cs
 - LoginViewModel.cs
 - Views
 - appsettings.json
 - Program.cs
- coreMvcFundamentals

Account.cs

coreFormsAndValidations

coreFormsAndValidations.Models.Account

Username

4

namespace coreFormsAndValidations.Models

5

{

6

0 references

7

public class Account

8

{

9

[Required]

10

[MinLength(5)]

11

[MaxLength(15)]

12

0 references

13

public string? Username { get; set; }

14

[Required]

15

0 references

16

public string? Password { get; set; }

17

0 references

18

public int Age { get; set; }

19

0 references

20

public string? Email { get; set; }

21

0 references

22

public string? Website { get; set; }

23

}

24

}

Solution Explorer

Search Solution Explorer (Ctrl+.)

Solution 'coreExperiencedDevelopersProjects' (2 of 2)

coreFormsAndValidations

Connected Services

Dependencies

Properties

wwwroot

Controllers

AccountController.cs

HomeController.cs

Models

Account.cs

ErrorViewModel.cs

LoginViewModel.cs

Views

appsettings.json

Program.cs

coreMvcFundamentals

Account.cs X

coreFormsAndValidations coreFormsAndValidations.Models.Account Email

```
4 namespace coreFormsAndValidations.Models
5 {
6     public class Account
7     {
8         [Required]
9         [MinLength(5)]
10        [MaxLength(15)]
11        public string? Username { get; set; }
12        [Required]
13        [MinLength(8)]
14        [MaxLength(15)]
15        public string? Password { get; set; }
16        [Range(18, 60)]
17        public int Age { get; set; }
18        [Required]
19        [EmailAddress]
20        public string? Email { get; set; }
```

Solution Explorer

Search Solution Explorer (Ctrl+;)

Solution 'coreExperiencedDevelopersProjects' (2)

- coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - AccountController.cs
 - HomeController.cs
 - Models
 - Account.cs
 - ErrorViewModel.cs
 - LoginViewModel.cs
 - Views
 - appsettings.json
 - C# Program.cs
 - coreMvcFundamentals

Account.cs X

coreFormsAndValidations coreFormsAndValidations.Models.Account

```
9      [MinLength(5)]
10     [MaxLength(15)]
      0 references
11     public string? Username { get; set; }
12     [Required]
13     [MinLength(8)]
14     [MaxLength(15)]
      0 references
15     public string? Password { get; set; }
16     [Range(18, 60)]
      0 references
17     public int Age { get; set; }
18     [Required]
19     [EmailAddress]
      0 references
20     public string? Email { get; set; }
21     [Url]
      0 references
22     public string? Website { get; set; }
23 }
24 }
```

AccountController.cs* Account.cs

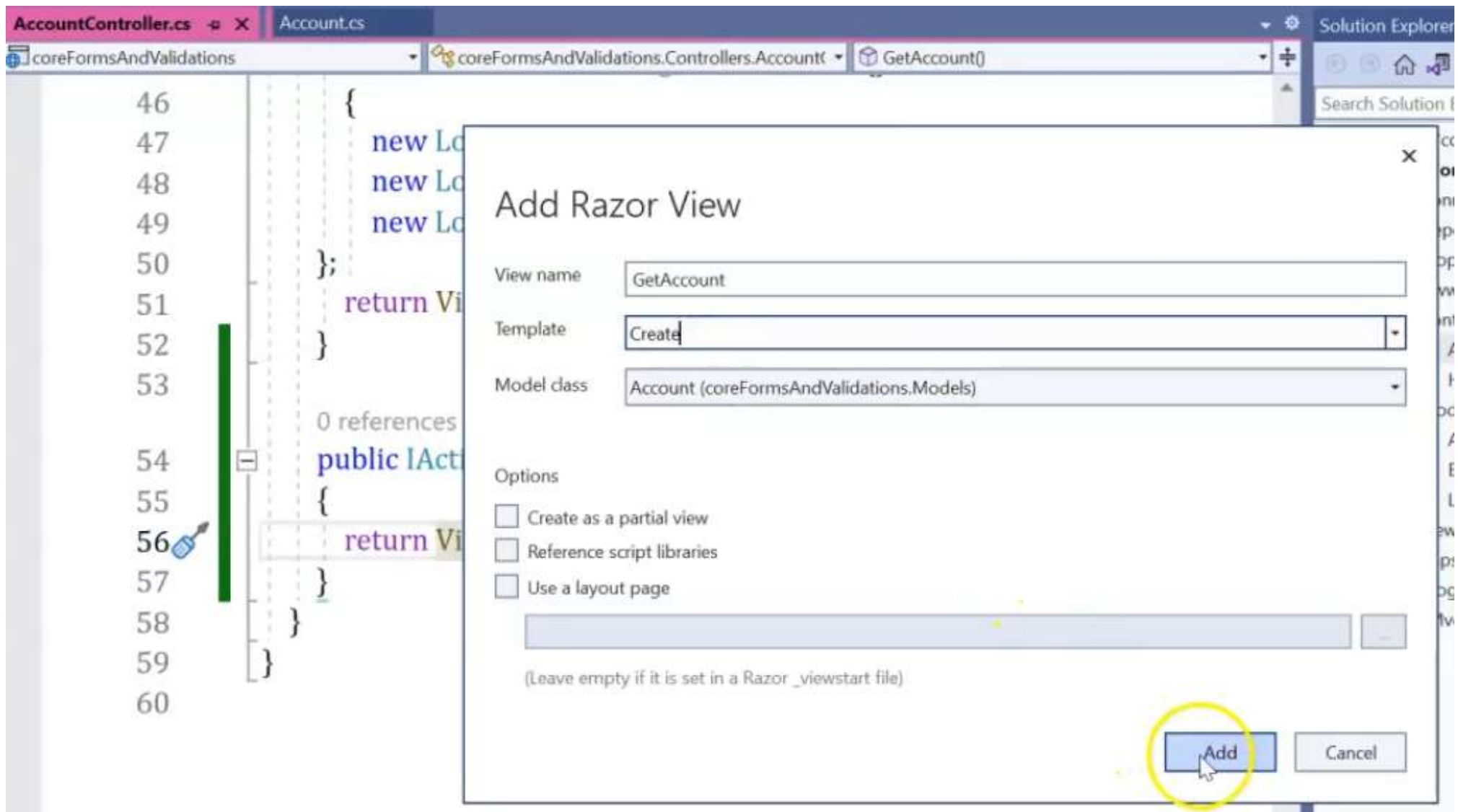
coreFormsAndValidations coreFormsAndValidations.Controllers.Account GetAccount()

```
46 {
47     new LoginViewModel() { Username = "Bhawna", Password = "12345"
48     new LoginViewModel() { Username = "Hitesh", Password = "123456"
49     new LoginViewModel() { Username = "Kaashvi", Password = "12345"
50 };
51 return View(users);
52 }
53
54 0 references
55 public IActionResult GetAccount()
56 {
57     return View();
58 }
59
60 }
```

Solution Explorer

Search Solution Explorer (Ctrl+;)

- Solution 'coreExperiencedDevelopersPro'
 - coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - C# AccountController.cs
 - C# HomeController.cs
 - Models
 - C# Account.cs
 - C# ErrorViewModel.cs
 - C# LoginViewModel.cs
 - Views
 - appsettings.json
 - C# Program.cs
 - coreMvcFundamentals



```
GetAccount.cshtml | AccountController.cs | Account.cs
10 <head>
11   <meta name="viewport" content="width=device-width" />
12   <title>GetAccount</title>
13 </head>
14 <body>
15
16   <h4>Account</h4>
17   <hr />
18   <div class="row">
19     <div class="col-md-4">
20       <form asp-action="PostAccount" method="post">
21         <div asp-validation-summary="ModelOnly" class="text-danger"></div>
22         <div class="form-group">
23           <label asp-for="Username" class="control-label"></label>
24           <input asp-for="Username" class="form-control" />
25           <span asp-validation-for="Username" class="text-danger"></span>
26         </div>
27         <div class="form-group">
28           <label asp-for="Password" class="control-label"></label>
29           <input asp-for="Password" class="form-control" />
30           <span asp-validation-for="Password" class="text-danger"></span>
31         </div>

```

GetAccount.cshtml | AccountController.cs | Account.cs

coreFormsAndValidations | coreFormsAndValidations.Controllers.AccountController | PostAccount(Account account)

```
52 }
53
54 public IActionResult GetAccount()
55 {
56     return View();
57 }
58
59 [HttpPost]
60 public IActionResult PostAccount(Account account)
61 {
62     if(ModelState.IsValid)
63     {
64         return View("Success");
65     }
66     return RedirectToAction("GetAccount");
67 }
68 }
69 }
```

0 references

0 references

Solution Explorer

Search Solution Explorer (Ctrl+)

Solution 'coreExperiencedDevelop...

- coreFormsAndValidations
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - AccountController.cs
 - HomeController.cs
 - Models
 - Account.cs
 - ErrorViewModel.cs
 - LoginViewModel.cs
 - Views
 - appsettings.json
 - Program.cs
- coreMvcFundamentals

Success.cshtml* x GetAccount.cshtml AccountController.cs Account.cs

1234567891011121314151617

@{
Layout = null;
}

<!DOCTYPE html>

<html>
<head>
 <meta name="viewport" content="width=device-width" />
 <title>Success</title>
</head>
<body>
 <h2>Home</h2>
</body>
</html>

Solution Explorer

Search Solution Explorer (Ctrl+.)

Solution 'coreExperiencedDevelo

coreFormsAndValidations

Connected Services

Dependencies

Properties

wwwroot

Controllers

AccountController.cs

HomeController.cs

Models

Account.cs

ErrorViewModel.cs

LoginViewModel.cs

Views

appsettings.json

Program.cs

coreMvcFundamentals



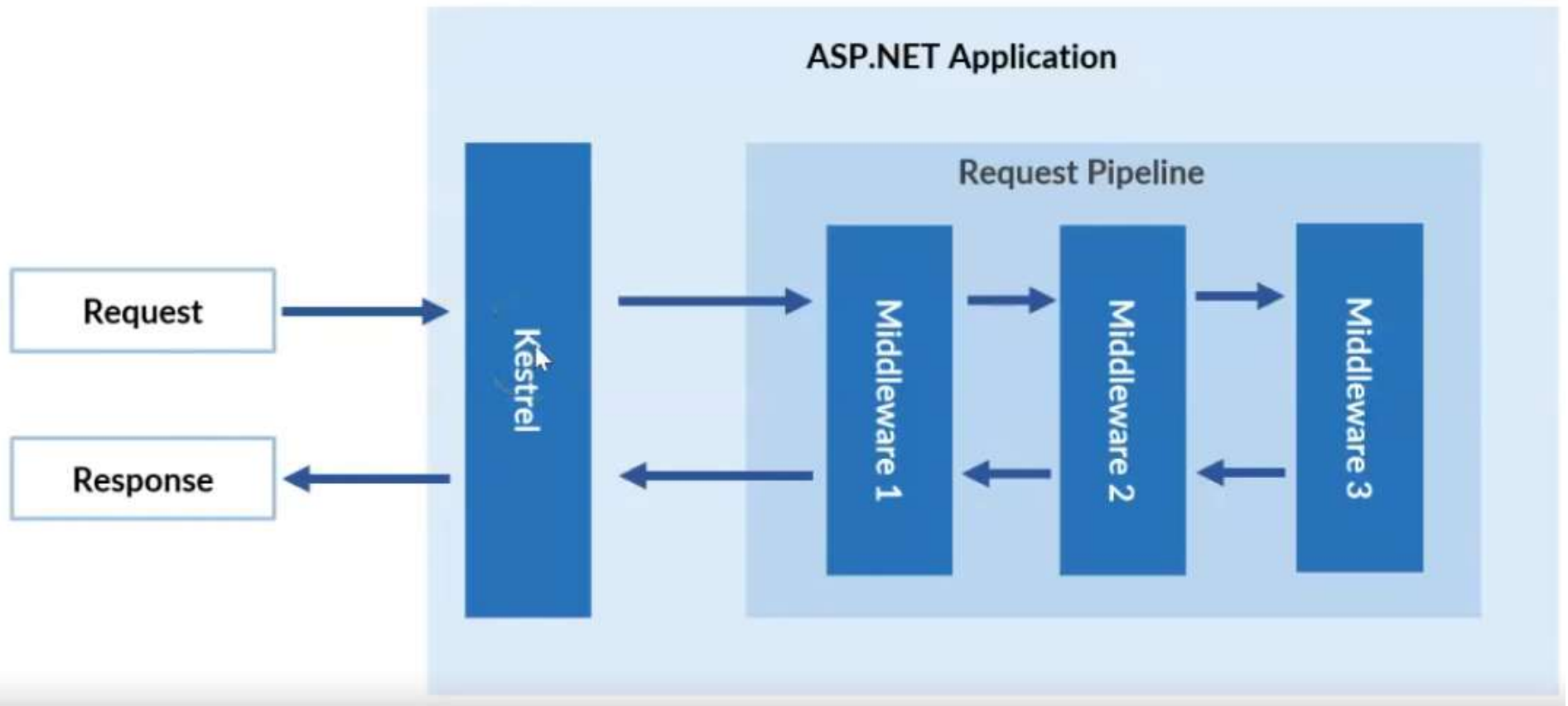
Advance concept in ASP .NET core

MIDDLEWARE REQUEST PIPELINE

Middle ware request pipeline

- In this, requests are processed beginning with request and ending with response.
- The pipeline specifies how the application should respond to the HTML request.
- The request arriving from the browser goes through the pipeline and back.
- The individual components that make up the pipeline are called Middleware.

MIDDLEWARE REQUEST PIPELINE



Middle ware request pipeline

- Authentication
- Static File
- Routing
- Session
- Exceptional Handling
- Cross-Origin Resource Sharing (CORS)
- HTTPS Redirection

How Middle ware request pipeline works

- First, the Http Request arrives (directly or via External web server) at the Application
- The kestrel Web Server picks up the Request.
- And creates the httpContext and passes it to the first middleware in the request pipeline.
- The first middleware then takes over, process request and passes it to the next Middleware.
- This goes on until it reaches the last middleware.
- The last one return the request back to previous middleware, terminating the request pipeline.
- Finally, the response reaches kestrel, which returns the response back to the client.
- Any of the middleware in the request pipeline can terminate the request pipeline by simply not passing the request to the next middleware.

**BUILT IN
MIDDLEWARE
AND CUSTOM
MIDDLEWARE**

BUILT-IN MIDDLEWARE



Middleware	Description
Authentication	Provide the authentication support for the application.
CORS	Used for configures Cross-Origin Resource Sharing.
Diagnostics	It is used to configures diagnostics
HTTPS Redirection	It is used to redirect all HTTP request to the HTTPS Service.
Request Localization	It provides the localization support.
Routing	This middleware provides the definition of the request routes.
HTTP Method Override	Provides us for the incoming POST request
MVC	Handles all the requests send or receive from the MVC or Razor pages.

Custom Middle Wares

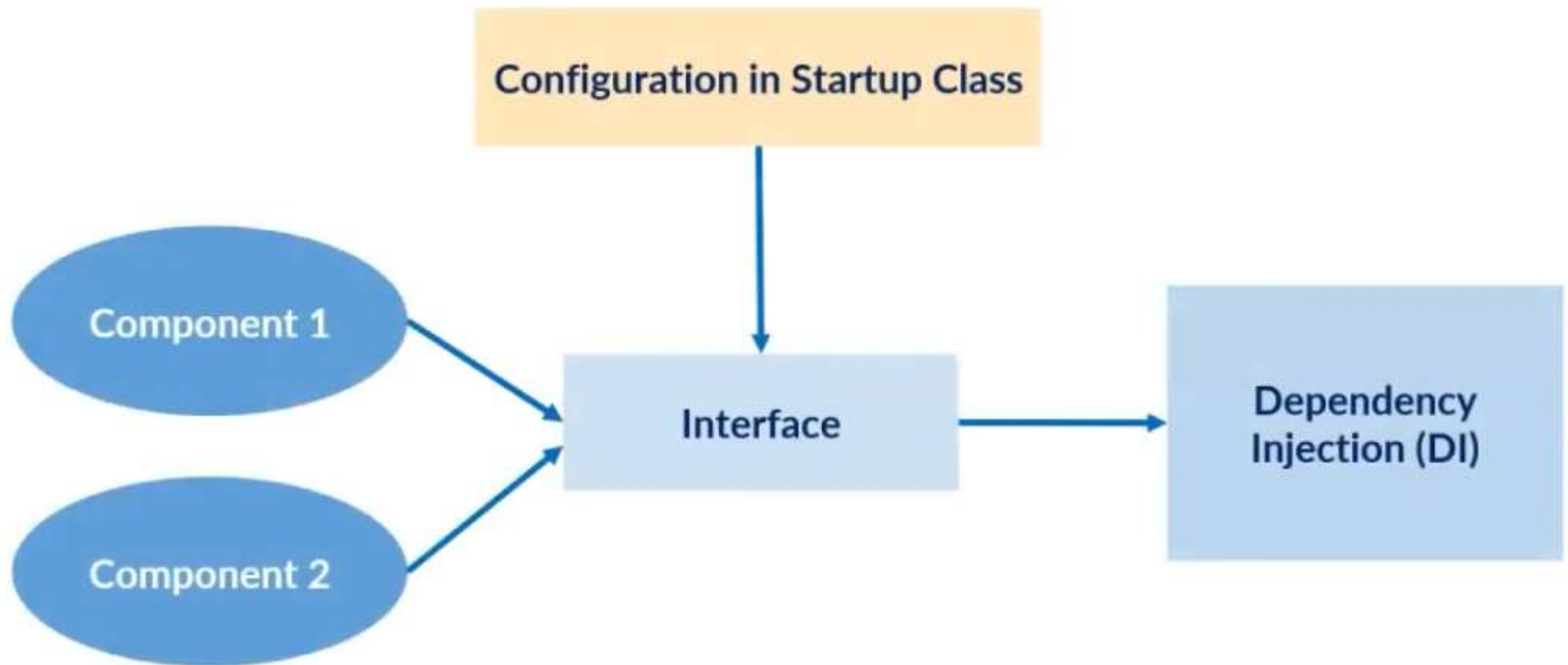
- The custom middleware component is like any other .NET class with `invoke()` method.
- However, in order to execute next middleware in a sequence, it should have `RequestDelegate` type parameter in the constructor.
- Custom middleware in .NET Core allows developers to execute code before or after request response cycle.

DEPENDENCY INJECTION IN .NET CORE

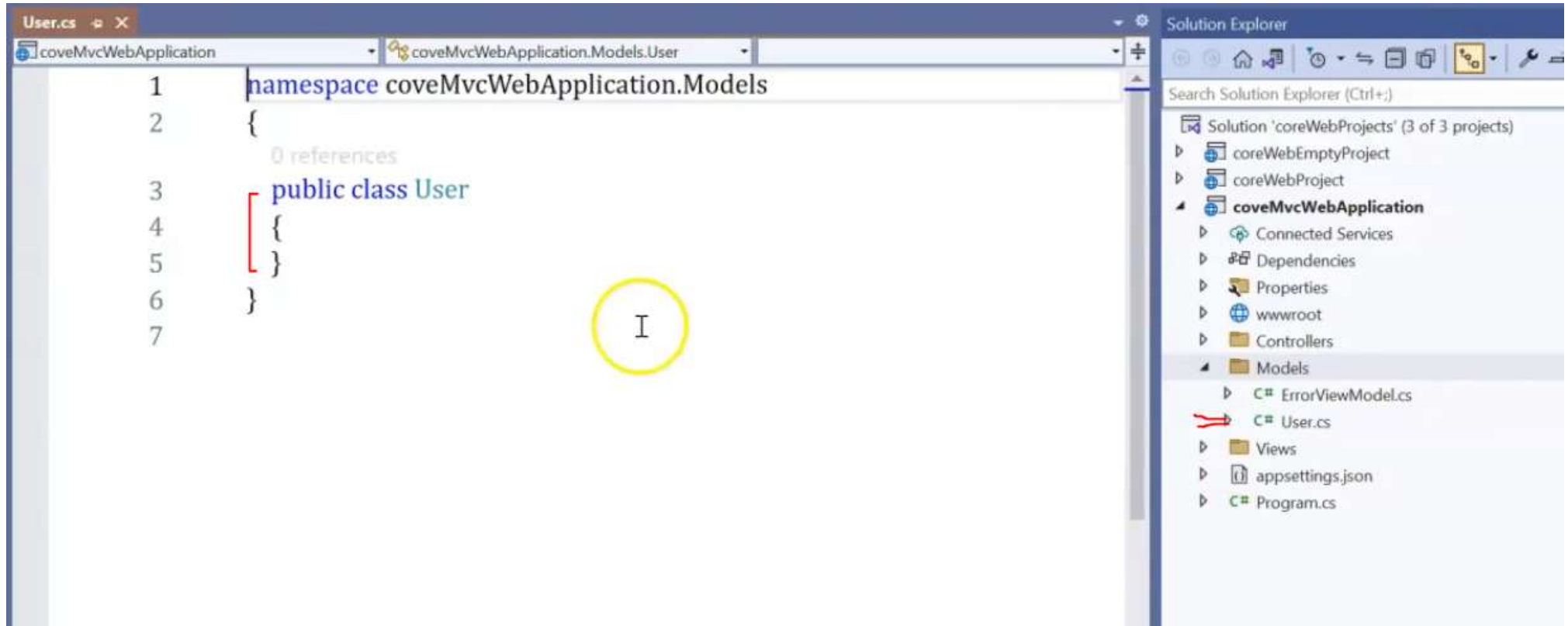
Dependency injection in ASP.NET Core

- DI is used to achieve loosely coupling between objects,
- So that applications can be maintained in an easy manner.
- Here, dot net runtime engine automatically injects objects of dependency classes.
- It injects mainly through the constructor of the controllers.
- Thus, making the job of the developer much easier.
- The concept of DI arises from a principle software design called dependency inversion principle.

DEPENDENCY INJECTION IN ASP.NET CORE



DEPENDENCY INJECTION IN ASP.NET CORE APPLICATIONS



User.cs

coveMvcWebApplication

coveMvcWebApplication.Models.User

Email

1

2

3

4

5

6

7

8

9

10

namespace coveMvcWebApplication.Models

{

0 references

public class User

{

0 references

public int Id { get; set; }

0 references

public string Name { get; set; }

0 references

public string Email { get; set; }

}

}

Solution Explorer

Search Solution Explorer (Ctrl+;)

Solution 'coreWebProjects' (3 of 3 projects)

coreWebEmptyProject

coreWebProject

coveMvcWebApplication

Connected Services

Dependencies

Properties

wwwroot

Controllers

Models

C# ErrorViewModel.cs

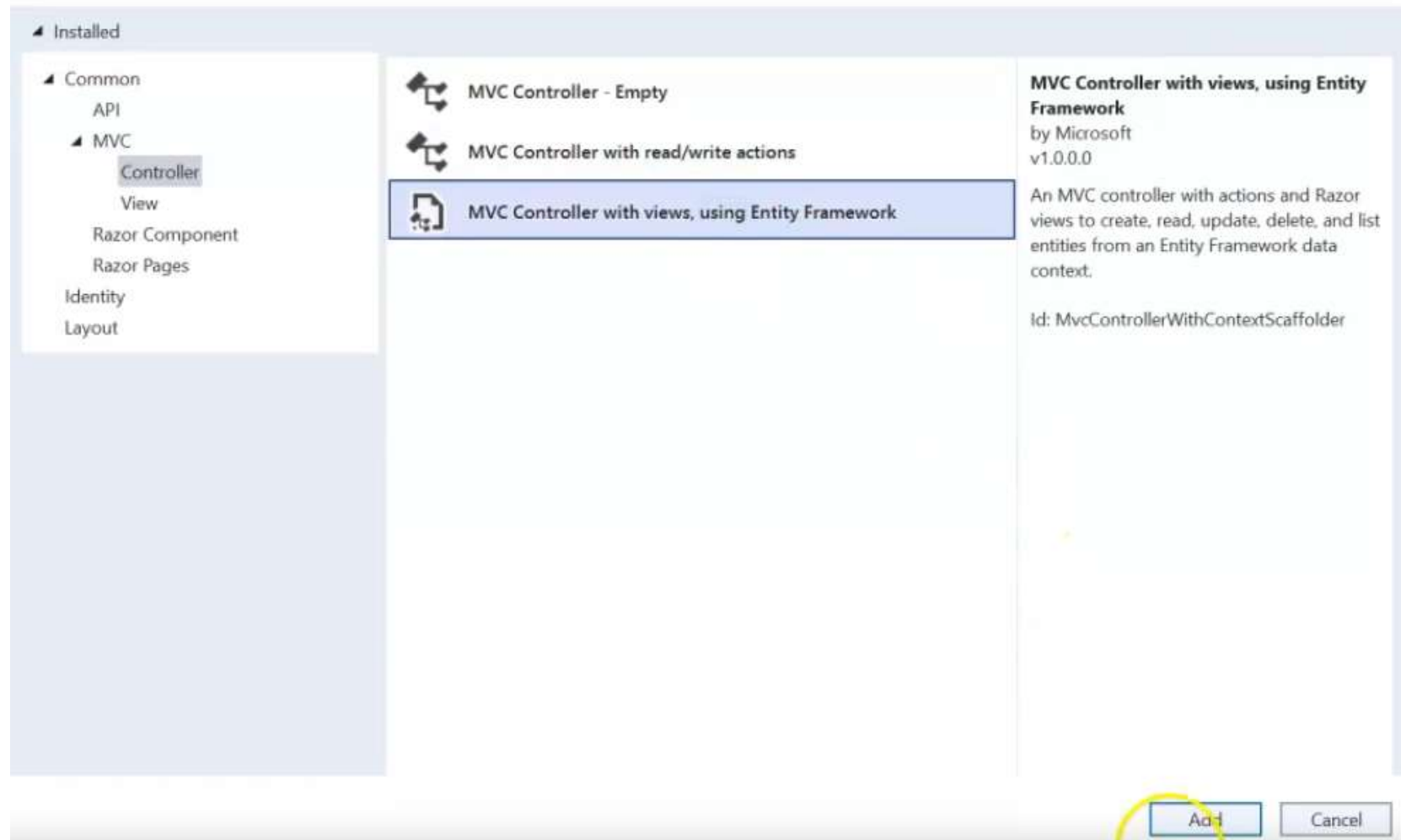
C# User.cs

Views

appsettings.json

C# Program.cs

Add New Scaffolded Item



Add MVC Controller with views, using Entity Framework

Model class

User (coveMvcWebApplication.Models)

Add Data Context

New data context type

coveMvcWebApplication.Data.coveMvcWebApplicationContext



Cancel

Controller name

UsersController

Add

Cancel

Add MVC Controller with views, using Entity Framework

Model class User (coveMvcWebApplication.Models) ▾

DbContext class coveMvcWebApplication.Data.coveMvcWebApplicationContext ▾ +

Views

☒ Generate views

☒ Reference script libraries

☒ Use a layout page

...

(Leave empty if it is set in a Razor _viewstart file)

Controller name UserController

Add Cancel

ASP.NET CORE ENVIRONMENT VARIABLES

ASP.NET Core environment variables

- ASP.NET Core environment variables uses the variable called ASPNETCORE-Environment.
- It points towards the runtime environment.
- The value of variable can be anything but usually be a production, staging, and development.
- The value is case insensitive in windows and Mac OS but it is case sensitive on Linux.
- The environment variable defines the runtime environment for which the application runs.
- They are mainly used to specify environment application executing, enabling the application to configure it.

ASP.NET Core environment variables

- Environment variables can be set to any of the values: Development, production, or staging.
- **Development:**
 - This process will be used during the application development.
 - While using visual studio, the setting will be denoted in project's debug, like IIS express.
- **Staging:**
 - It is Pre-production environment used for testing after the application before deployment.
- **Production:**
 - The production environment is used while the application executes during the live process where the end-user works on it.

DEPENDENCY INJECTION IN ASP.NET CORE APPLICATIONS

User.cs

coveMvcWebApplication

coveMvcWebApplication.Models.User

Email

```
1 namespace coveMvcWebApplication.Models
2 {
3     0 references
4     public class User
5     {
6         0 references
7         public int Id { get; set; }
8         0 references
9         public string Name { get; set; }
10        0 references
11        public string Email { get; set; }
12    }
13 }
```

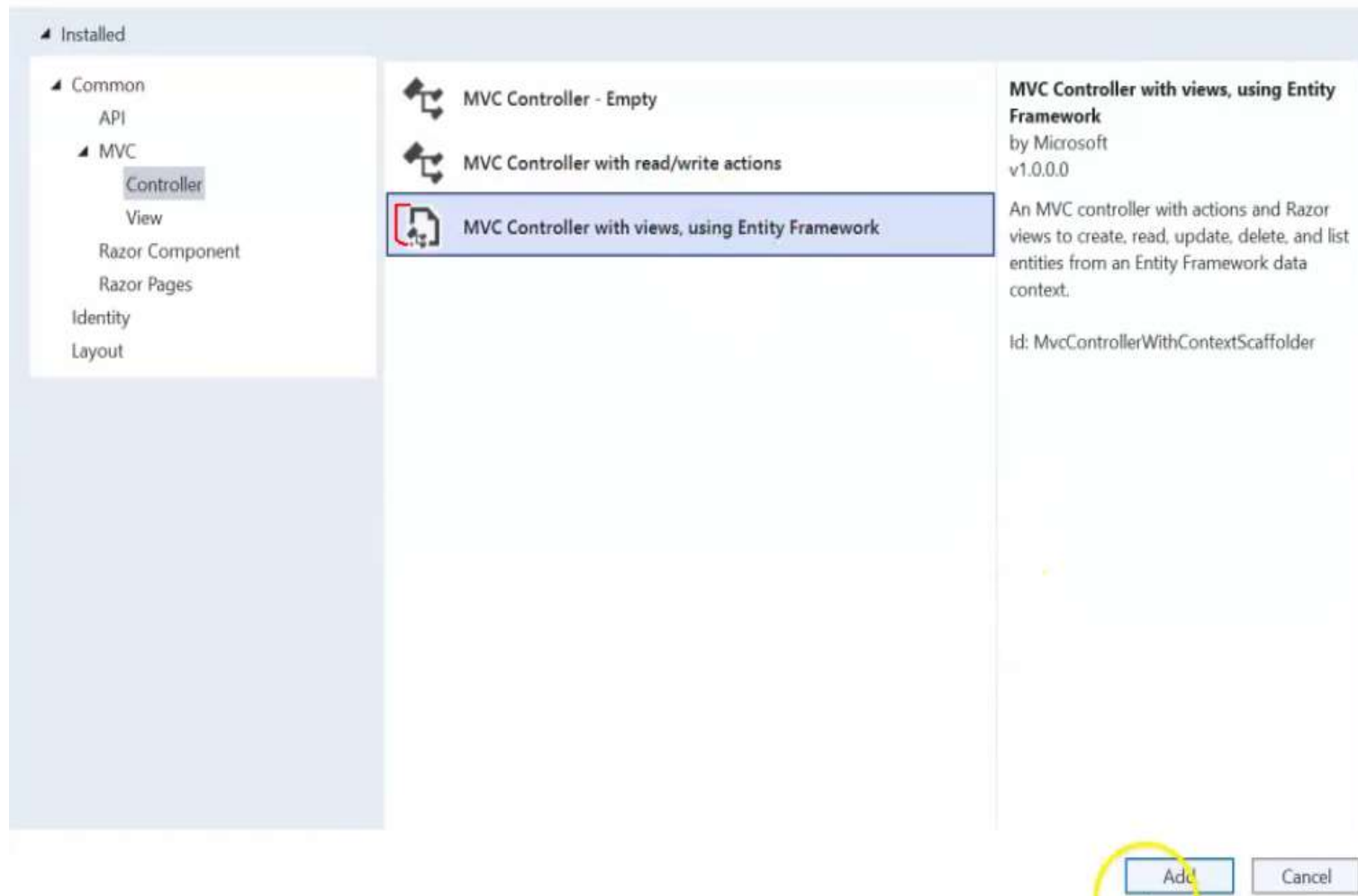
Solution Explorer

Search Solution Explorer (Ctrl+;)

Solution 'coreWebProjects' (3 of 3 projects)

- coreWebEmptyProject
- coreWebProject
- coveMvcWebApplication**
 - Connected Services
 - Dependencies
 - Properties
 - wwwroot
 - Controllers
 - Models**
 - C# ErrorViewModel.cs
 - C# User.cs**
 - Views
 - appsettings.json
 - C# Program.cs

Add New Scaffolded Item





Add MVC Controller with views, using Entity Framework

Model class User (coveMvcWebApplication.Models)

DbContext class coveMvcWebApplication.Data.coveMvcWebApplicationContext +

Views

- ☒ Generate views
- ☒ Reference script libraries
- ☒ Use a layout page

...

(Leave empty if it is set in a Razor _viewstart file)

Controller name UsersController

Add

Cancel

coveMvcWebA...nContext.cs appsettings.json Program.cs **UsersController.cs** User.cs

coveMvcWebApplication coveMvcWebApplication.Controllers.UsersController UsersController(coveMvc)

```
10
11 namespace coveMvcWebApplication.Controllers
12 {
13     1 reference
14     public class UsersController : Controller
15     {
16         private readonly coveMvcWebApplicationContext _context;
17
18         0 references
19         public UsersController(coveMvcWebApplicationContext context)
20         {
21             _context = context;
22         }
23
24         // GET: Users
25         3 references
26         public async Task<IActionResult> Index()
27         {
```

Exception handling in .NET Core

- Exception handling is one of the important tasks in the application development cycle.
- **Below are the effective ways to handle the exceptions in .NET Core.**
 - Exception Handling with try-catch block
 - Global exception Handling with custom Middleware

Exception handling with try-catch block

- The try block contains the code that may throw an exception.
- And the catch block catches the exception and provides a way to handle it.
- The finally block is optional and contains code that will be executed regardless of whether an exception is thrown or not.
- `Public IActionResult index(){`
- `Try`
- `{`
- `}catch(exception e) {..}`
- `}`

Imagine that you have many controllers and actions in your project and then you need to use try-catch for every action in controllers. some times you have to use try-catch in the services. In that case, it increases the lines of code your project and it is not good.

This Problem can be handled by
Global Exception Handling with Custom Middleware

Exception handling with custom middle ware

- This is a good and effective approach to handling the exception global level.
- We could be caught all unhandled exceptions using this exception handler.
- The benefit of using this approach is we could be caught all exceptions in one place.
- And don't need to use try-catch in every action to catch errors.
- In that case, we need to create custom middleware to handle the exceptions.

Introduction to ORM

- ORM stands for object-relational mapping.
- Object-relational mapping (ORM) is a way to align programming code with database structures.
- ORM uses metadata descriptions to create a layer between the programming language and a relational database.
- It thus connects OOPs code with the database.
- And simplifiers the interaction between relational databases and OOP languages.

How ORM works?

- The whole methodology followed by ORMs is dependent on the object-oriented
- ORMs generate objects which map to tables in the database virtually.
- Once these objects are up, then coders can easily work to retrieve. Manipulate or delete any field from the table without paying much attention to language specifically.
- It supports writing complex long SQL queries in a simpler way.

How ORM works?

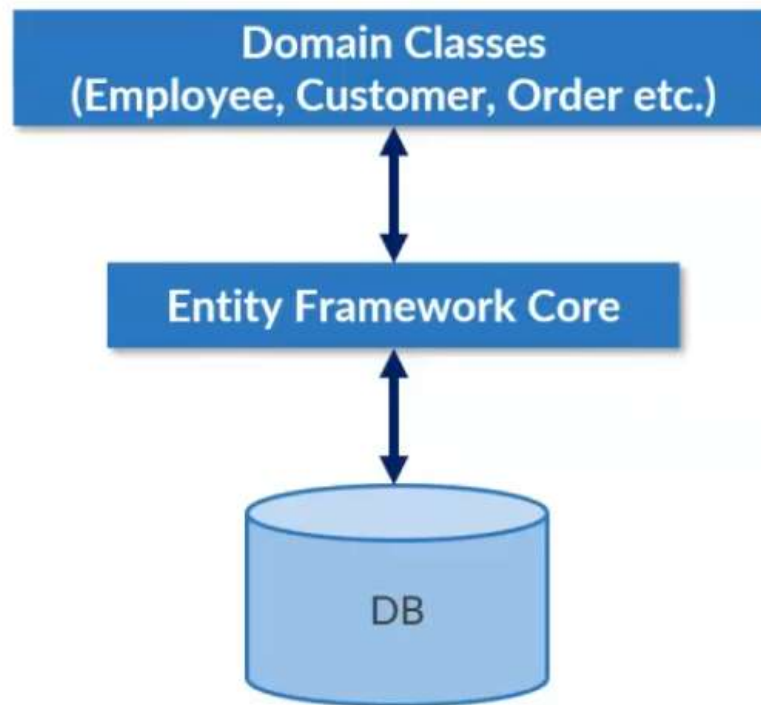
- Suppose one is using to link python application with oracle database.
- In that case, the developer will have to choose the most relevant ORM as per the business requirements.
- This can be well understood by the diagram below:



Introduction to EF Core

- Entity Framework is an Object Relational Mapping (ORM) framework.
- Entity framework Core is a lightweight and extensible version of Entity Framework.
- It enables us to work with databases using .NET objects.
- EF Core provides a set of APIs for querying and manipulating data in relational databases.
- It supports a variety of database providers, including Microsoft SQL Server, SQLite, MySQL, PostgreSQL, and others.

DOMAIN CLASSES



Types of EF Core

- **Code first Approach:**

Code first approach works based on Domain Driven Design. In Code First approach,

- Create Domain Classes.
- Create DB Context class derived from EF Core Db context classes.
- EF Core create Db and Tables using a default configuration.
- You can change the EF Core default configuration.



Types of EF Core

- **Database first Approach:**

The database approach will help us to create Domain classes and DB Context classes from an existing database using EF Core.

If you have an existing Database then we should use this approach to work with ORM.



CONFIGURING ENTITY FRAMEWORK CORE IN MVC APPLICATION

